

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/374069161>

PORTFOLIO OPTIMIZATION with REINFORCEMENT LEARNING

Thesis · June 2023

DOI: 10.13140/RG.2.2.31438.23363

CITATIONS

0

READS

604

2 authors, including:



Eren Kotar

Istanbul Technical University

1 PUBLICATION 0 CITATIONS

SEE PROFILE

ISTANBUL TECHNICAL UNIVERSITY ★ FACULTY OF MANAGEMENT

PORTFOLIO OPTIMIZATION with REINFORCEMENT LEARNING

B.Sc. THESIS

**Eren KOTAR
İnci KINALI**

Department of Industrial Engineering

JUNE 2023

ISTANBUL TECHNICAL UNIVERSITY ★ FACULTY OF MANAGEMENT

PORTFOLIO OPTIMIZATION with REINFORCEMENT LEARNING

B.Sc. THESIS

**Eren KOTAR
(070190163)**

**İnci KINALI
(070190747)**

Department of Industrial Engineering

Thesis Advisor: Hüseyin Kutay TİNÇ, PhD

JUNE 2023

İSTANBUL TEKNİK ÜNİVERSİTESİ ★ İŞLETME FAKÜLTESİ

PEKİŞTİRMELİ ÖĞRENME İLE PORTFÖY OPTİMİZASYONU

LİSANS TEZİ

Eren KOTAR

070190163

İnci KINALI

070190747

Endüstri Mühendisliği Anabilim Dalı

Tez Danışmanı: Arş.Gör.Dr. Hüseyin Kutay Tinç

HAZİRAN 2023

To our families and friends,

FOREWORD

Throughout our academic journey, we have embraced the fascinating fields of data science and machine learning, recognizing their strong correlation with industrial engineering in terms of quantitative decision-making, mathematical modeling, and optimization.

In today's data-driven world, the ability to extract meaningful insights and make informed decisions is crucial. We have delved into the realm of data science, particularly focusing on machine learning techniques, as powerful tools for understanding complex phenomena and deriving valuable knowledge from data. This interdisciplinary approach has allowed us to explore the intersection of industrial engineering with data-driven decision-making.

We would like to thank Prof. Vehbi Tufan Koç for laying the foundations of the manufacturing field in industrial engineering context, Prof. Yusuf İlker Topçu and Prof. Özgür Kabak for their comprehensive insights into operations research field, Assoc. Prof. Umut Asan for his in-depth explanations on statistics leading up to philosophy and the vision he added, Assist. Prof. Mehmet Ali Egün and Assist. Prof. Mehmet Yasin Ulukuş for their very systematic and enthusiastic explanations about data analysis and algorithmic way of thinking, Assist. Prof. Ömer Faruk Beyca for his contribution to the understanding of machine learning and the application side of it, and Assoc. Prof. Nazım Kemal Üre from computer science faculty for warmly welcoming us into his class and providing us with invaluable insights into the field of deep reinforcement learning.

Lastly, we would like to express our sincere thanks and respect to our thesis advisor, Res. Assist. Prof. Kutay Tinç, who supported us in every field during our university life and mentored our education and training.

Their unwavering dedication and insightful guidance have significantly shaped the direction of our thesis and enriched our understanding of industrial engineering and discipline.

June 2023

Eren Kotar
Industrial Engineer
İnci Kınalı
Industrial Engineer

TABLE OF CONTENTS

	<u>Page</u>
FOREWORD	vii
TABLE OF CONTENTS	ix
ABBREVIATIONS	xiii
SYMBOLS	xv
LIST OF TABLES	xvii
LIST OF FIGURES	xix
SUMMARY	xxi
ÖZET	xxiii
1. INTRODUCTION	1
1.1 Purpose of Thesis	2
2. SYSTEM ANALYSIS	3
2.1 Entities	4
2.2 Environment	5
2.3 Constraints	5
2.4 Assumptions	5
2.5 Parameters	5
2.6 Objective	5
3. LITERATURE REVIEW	7
3.1 Portfolio Management	7
3.1.1 Financial portfolio management	7
3.1.2 Portfolio optimization	8
3.1.3 Stock trading	8
3.1.3.1 Stock trading with machine learning	10
3.2 Machine Learning	11
3.2.1 Supervised learning	12
3.2.2 Unsupervised learning	12
3.2.3 Deep learning	13
3.2.3.2 Neural networks	13
3.2.3.1 Artificial neural networks (ANN)	14
3.2.3.2 Recurrent neural networks (RNN)	15
3.2.3.3 Long-short-term memory (LSTM)	15
3.2.4 Reinforcement learning	16
3.2.4.1 Markov chains	17
3.2.4.2 Markov decision processes	17
3.2.4.3 Reward function	18
3.2.4.4 Discount factor (γ)	18
3.2.4.5 Value function	18
3.2.4.6 Policy (π)	19
3.2.4.7 Bellman equation	19
3.2.4.8 Optimal policy and value iteration	19

3.2.4.9 Q-learning.....	20
3.2.4.10 Epsilon greedy discovery	20
3.2.4.11 Function approximation	21
3.2.4.12 Deep reinforcement learning.....	21
3.2.4.13 Explore method	21
3.2.4.14 Exploit method	22
3.2.4.15 Data augmentation.....	22
3.2.4.16 Experience replay	22
3.2.4.17 Entropy based discovery	23
3.2.4.18 Actor-critic (A2C)	23
3.2.4.19 Proximal policy optimization (PPO).....	24
3.3 Natural Language Processing	25
3.4 Time Series Analysis	25
4. METHODOLOGY	27
4.1 Python.....	27
4.1.1 Numpy	27
4.1.2 Pandas.....	28
4.1.3 Matplotlib	28
4.1.4 Scikit-learn	28
4.1.5 Tensorflow.....	29
4.1.6 Gym	30
4.1.7 Yahoo finance.....	30
4.1.8 Stable-Baselines3	30
4.1.9 FinRL.....	30
4.2 Technical Analysis	30
4.3 Technical Indicators	31
4.3.1 Sharpe ratio.....	31
4.3.2 Moving average	32
4.3.3 Relative strength index (RSI)	33
4.3.4 Commodity index channel (CCI)	34
4.3.5 Moving average convergence/divergence (MACD)	35
4.3.6 Directional movement index (DX).....	36
4.3.7 Larry williams R%	37
4.4 Evaluation.....	37
4.4.1 Splitting	38
4.4.2 Approximation error.....	38
4.4.3 Benchmarking	38
5. APPLICATION.....	39
5.1 Modules & Functions	39
5.2 Data.....	39
5.2.1 Stock selection.....	39
5.2.2 Data extraction	40
5.2.3 Data split.....	40
5.2.4 Data analysis.....	41
5.2.5 Adding technical indicators.....	43
5.3 Model Design	43
5.3.1 Overview	44
5.3.2 Environment design.....	44
5.3.2.1 Observation space	45
5.3.2.2 Action space	45

5.3.2.3 Reward function	46
5.3.2.4 Initialising	48
5.3.2.5 Policy optimization method	48
5.4 Model Training.....	49
5.5 Model Evaluation During Training	49
5.5.1 KL divergence approximatation.....	50
5.5.2 Clip fraction	50
5.5.3 Clip range	50
5.5.4 Entropy loss.....	50
5.5.5 Explained variance	50
5.5.6 Learning rate	51
5.5.7 Loss	51
5.5.8 Number of updates	51
5.5.9 Policy gradient loss	51
5.5.10 Reward	51
5.5.11 Standard deviation (std)	51
5.5.12 Value loss	51
5.6 Testing / Evaluation	52
5.6.1 Actions	52
5.6.2 Compharison between benchmarks.....	53
5.6.2.1 Annual return	54
5.6.2.2 Cumulative return	55
5.6.2.3 Annual volatility	55
5.6.2.4 Sharpe ratio	55
5.6.2.5 Calmar ratio.....	56
5.6.2.6 Stability	56
5.6.2.7 Max drawdown	56
5.6.2.8 Omega ratio.....	56
5.6.2.9 Sortino ratio.....	56
5.6.2.10 Skew.....	56
5.6.2.11 Kurtosis	56
5.6.2.12 Tail ratio	57
5.6.2.13 Daily value at risk (VaR)	57
5.7 Tool Interface	57
6. CONCLUSIONS AND RECOMMENDATION	63
7. REFERENCES.....	65
CURRICULUM VITAE.....	71

ABBREVIATIONS

ML	: Machine Learning
AI	: Artificial Intelligence
ANN	: Artificial Neural Networks
API	: Application Programming Interface
DL	: Deep Learning
CNN	: Convolutional Neural Networks
LSTM	: Long-short-term Memory
RNN	: Recurrent Neural Networks
RSI	: Relative Strength Index
RS	: Relative Strength
MACD	: Moving Average Convergence Divergence
MDP	: Markov Decision Process
NLP	: Natural Language Processing
CWI	: Centrum voor Wiskunde en Informatica
OO	: Object-oriented
MSE	: Mean Squared Error
SMA	: Simple Moving Average
EMA	: Exponential Moving Average
CCI	: Commodity Index Channel
FX	: Foreign Exchange
MDP	: Markov Decision Processes
RL	: Reinforcement Learning
PPO	: Proximal Policy Optimization
DX	: Directional Movement Index
KPI	: Key Performance Indicator
APPL	: Apple's ticker name
NFLX	: Netflix's ticker name
GOOG	: Google's ticker name
ORCL	: Oracle's ticker name
MSFT	: Microsoft's ticker name
IBM	: International Business Machines' ticker name

SYMBOLS

S_i	: I th state of the system
γ	: Discount factor (gamma)
α	: Learning rate (alpha)
r	: Reward
π	: Policy
t	: Time
M	: Number of stocks in portfolio
i	: Time set
j	: Stock set
SO_{ij}	: Number of shares owned at the time i of stock j in portfolio
SV_{ij}	: Shares values at the time i of stock j in portfolio
PV_i	: Portfolio value at the time i
R_f	: Risk free rate
R_p	: Return of Portfolio
σ_f	: Standart deviation of the portfolio's excess return
$\$$	: Dollar

LIST OF TABLES

	<u>Page</u>
Table 3.1 : Order book before (a).....	10
Table 3.2 : Order book before (b).	10
Table 5.1 : Stocks included in portfolio.	39
Table 5.2 : First 7 instances of data.....	40
Table 5.3 : Common metrics of stocks selected from 2008-2018.....	42
Table 5.4 : Table consists of added technical indicators.....	43
Table 5.5 : Allocations of each stock through time.....	52
Table 5.6 : Benchmark results.....	54

LIST OF FIGURES

	<u>Page</u>
Figure 3.1 : Representation of neuron system.....	14
Figure 3.2 : Recurrent neural network architecture.	15
Figure 3.3 : LSTM architecture.....	16
Figure 3.4 : Visual representation of markov chain with nodes and arcs.	17
Figure 3.5 : Transition probability matrix.....	17
Figure 3.6 : Deep reinforcement learning architecture.	21
Figure 3.7 : Actor-Critic architecture.....	24
Figure 4.1 : CCI in a stock chart.	35
Figure 4.2 : Different sets of data.	38
Figure 5.1 : Importing necessary functions from modules.	39
Figure 5.2 : Code snippet demonstrating the data extraction process.....	40
Figure 5.3 : Data splitting process.	41
Figure 5.4 : Stock chart.	41
Figure 5.5 : Line plot of stocks returns between 2008-2018.	42
Figure 5.6 : Adding indicators.	43
Figure 5.7 : Architecture of PPO method.	44
Figure 5.8 : Class that defines environment.....	44
Figure 5.9 : Attributes defined in environment class.....	45
Figure 5.10 : Defining observation space attribute.	45
Figure 5.11 : Defining action space.	46
Figure 5.12 : Reward function.	47
Figure 5.13 : Initializing environment's parameters.	48
Figure 5.14 : Initilaising RL agent's parameters.	48
Figure 5.15 : Training of RL agent.	49
Figure 5.16 : Performance metric at iteration (trade) 528 and 18293.....	49
Figure 5.17 : Performance metrics at batch 2, 40, and 500 (final batch).	50
Figure 5.18 : Box plot of allocations.	52
Figure 5.19 : Graphical representation of allocations through time.	53
Figure 5.20 : Creating summary metrics by "timeseries.perf_stats" function.	54
Figure 5.21 : Cumulative return graph though time.....	55
Figure 5.22 : Flow chart of tool (a).....	57
Figure 5.23 : Flow chart of tool (b).....	58
Figure 5.24 : Homepage of tool	58
Figure 5.25 : Analysis page	59
Figure 5.26 : Model page.....	60
Figure 5.27 : Dashboard page.	61

PORTFOLIO OPTIMIZATION with REINFORCEMENT LEARNING

SUMMARY

Money, investments, debt, and wealth management are just a few of the issues that are covered by the field of finance, which is crucial to the global economy. Growth and sustainability depend on firms, people, and governments using financial resources wisely. Risk management, valuation, portfolio management, capital markets, and an understanding of financial instruments are all aspects of finance. Among these fields, portfolio management became a crucial tool for investors. Portfolio optimization is a method to manage a portfolio and it has the ability to reduce risk, boost returns, and assist investors in making more knowledgeable investment selections. Stocks play a significant role in a portfolio and are quite vital to investors. As a result, choosing stocks and allocating a portfolio is a crucial decision-making process for investors. In recent years, the field of stock trading has paid a lot of attention to reinforcement learning, a crucial machine learning technique. Artificial intelligence technology can significantly benefit investors in the stock trading process, which involves making decisions in a dynamic and complex market. By interacting with its surroundings, an agent can learn to optimize a certain goal through reinforcement learning. This objective in stock trading is often to maximize profits or reduce risk. For difficult tasks like assessing market patterns, forecasting price changes, and creating risk management plans, reinforcement learning can be applied.

In the first part of this study, the system analysis and literature review of portfolio management and reinforcement learning, and methodology of this study will be mentioned. In the second part, application of the tool will be explained by considering data analysis, model design and interface.

In this article, a product which enables customer to trade stocks with reinforcement learning will be explained from different aspects, such as how it works, the advantages,

and potential challenges. At the same time, this study aims to explore the opportunities this tool can offer for investors and its impact on the financial markets.

PEKİŞTİRMELİ ÖĞRENME İLE PORTFÖY OPTİMİZASYONU

ÖZET

Para, yatırımlar, borç ve servet yönetimi, küresel ekonomi için çok önemli olan finans alanının ele aldığı konulardan sadece birkaçıdır. Büyüme ve sürdürülebilirlik, finansal kaynakları akıllıca kullanan şirketlere, insanlara ve hükümetlere bağlıdır. Risk yönetimi, değerlendirme, portföy yönetimi, sermaye piyasaları ve finansal araçların anlaşılması, finansın alanlarıdır. Bu alanlar arasında portföy yönetimi yatırımcılar için çok önemli bir araç haline gelmiştir. Portföy optimizasyonu, bir portföyü yönetmenin bir yöntemidir ve riski azaltma, getirileri artırma ve yatırımcılara daha bilgili yatırım seçimleri yapmalarında yardımcı olma yeteneğine sahiptir. Hisse senetleri bir portföyde önemli bir rol oynar ve yatırımcılar için oldukça hayati öneme sahiptir. Sonuç olarak, hisse senedi seçimi ve portföy tahsisi yatırımcılar için çok önemli bir karar verme sürecidir. Son yıllarda, hisse senedi ticareti alanı, çok önemli bir makine öğrenimi tekniği olan pekiştirmeli öğrenmeye çok dikkat etti. Yapay zeka teknolojisi, dinamik ve karmaşık bir piyasada karar vermeyi içeren hisse senedi alım satım sürecinde yatırımcılara önemli ölçüde fayda sağlayabilir. Etmen, çevresiyle etkileşim kurarak, pekiştirmeli öğrenme yoluyla belirli bir hedefi optimize etmeyi öğrenebilir. Hisse senedi alım satımındaki bu amaç genellikle karı maksimize etmek veya riski azaltmaktır. Piyasa modellerini değerlendirmek, fiyat değişikliklerini tahmin etmek ve risk yönetimi planları oluşturmak gibi zor görevler için pekiştirmeli öğrenme uygulanabilir.

Bu çalışmanın ilk bölümünde portfolyo yönetimi ve pekiştirmeli öğrenmenin sistem analizi ve literatür taramasından ve bu çalışmanın metodolojisinden bahsedilecektir. İkinci bölümde veri analizi, model tasarımı ve arayüzü ele alınarak aracın uygulaması anlatılacaktır.

Bu yazıda, pekiştirmeli öğrenme ile müşterilerin hisse senedi alıp satmalarını sağlayan bir ürün, nasıl çalıştığı, avantajları ve olası zorlukları gibi farklı yönleriyle açıklanacaktır. Aynı zamanda bu çalışma, bu aracın yatırımcılara sunabileceği fırsatları ve finansal piyasalar üzerindeki etkisini keşfetmeyi amaçlamakta

1. INTRODUCTION

Portfolio management has been a subject that people have been interested in and researched for many years. In order to create a successful portfolio by choosing the right investment, it is necessary to manage the portfolio correctly by making the stock estimation with the highest accuracy. Due to the inherent uncertainty, predicting stocks and stock price indices is challenging. Before buying a stock, investors conduct two different sorts of analysis. The fundamental analysis comes first. Investors use this to determine if they should invest by looking at the intrinsic worth of stocks, the performance of the sector and economy, the political environment, etc. The appraisal of equities using technical analysis, in addition, involves looking at market data such as historical prices and volume. Technical traders analyse stock charts to spot trends and patterns that can hint at how a stock may respond in the future rather than attempting to calculate a security's fundamental value (Patel et al., 2015). Nowadays, in an effort to clarify many peculiar market phenomena, financial regulators as well as the media have concentrated their focus on algorithmic trading (Nutti et al., 2011). Among all approaches, reinforcement learning is more suitable due to several reasons such as the ease of adaptability, decision-making under uncertainty, exploration and exploitation, complex pattern recognition and optimization of trading strategies. That's why this paper will use reinforcement learning to solve the portfolio optimization problem.

In recent era, portfolio optimization with reinforcement learning is a significant issue as it enables investors to adapt to dynamic market conditions, handle complex data, capture non-linearities, incorporate risk management, and strike a balance between exploration and exploitation. In the financial world which has rocketing changes, these skills may result in enhanced portfolio performance and more consistency with investor goals. This thesis aims to solve a portfolio optimization problem by using reinforcement learning approaches and create a product which enables customer to trade with maximum return.

1.1 Purpose of Thesis

The objective of this graduation thesis is to create a portfolio management tool for multi-stock trading that utilizes reinforcement learning techniques. The primary aim is to optimize the value of the customer's portfolio by accurately allocating the appropriate amounts to the portfolio assets, which primarily include stocks.

Additionally, this study aims to provide comprehensive analysis and evaluation of the decision-making process to facilitate informed decision making. By conducting a thorough analysis of the outcomes, the study aims to enhance the understanding of the results and their implications for effective decision making to the user.

2. SYSTEM ANALYSIS

The problem of the system is able to be summarized as gaining profit. In detail, the purposes of this system are striving to seize momentary price anomalies, gaining profit from statistical trends placed in financial markets, executing orders as efficiently as possible, concealing a trader's objectives, or identifying and taking advantage of competitors' tactics (Nuti et al., 2011). The trading orders placed by the traders in the system must match. What determines the dynamics of the system is the stakeholders themselves within the system. The buy orders they place increase the share price while the sell orders decrease the price of the stock. These are the potential opportunities and threats of the system. There are many indicators in the stock trading system. These indicators give information about the trend of the stock and are tools used in price prediction. In addition to these, there are numerous parameters in stock market systems. Stock market systems, which can be affected by many tangible and intangible changes, are very sensitive and this causes fluctuations in share prices. These are the factors that complicate stock price estimation and thus constrain the system. system design is based on predicting share prices using machine learning. For this purpose, different machine learning techniques in the literature will be combined and it will be tried to predict share prices with the highest possible accuracy. The study will provide the stakeholders with the opportunity to easily trade alogrhythmic stock exchanges and earn high profits. The aim of the designed system is to maximize the level of accuracy. The share price predictions of a model with a high level of accuracy will be close to the truth and investors will be able to earn high profits. The forecasting or optimization part of the system has no direct effect on the system. In other words, making accurate forecasts does not imply that making good trades. For this reason, two separate success criteria and the correlation between these success criteria will be taken into account to measure the overall success of the system.

The goal of this project is to design a portfolio management tool using machine learning approaches to optimize stock selection decisions. The tool will use machine

learning, specifically reinforcement learning, to decide whether it's best to buy, sell, or hold stocks in a given portfolio.

Our system as a product, will provide the customer with a comprehensive analysis of the outputs after the stocks have been selected and the model has been established, allowing for selection and analysis based on previous data prior to the stock selection process. The customer will encounter a training cost and duration of the train according to the parameters (train days, iterations, number of shares in the portfolio, etc.). With this process, a part of the cost of the system will be collected from the customer and converted into a benefit. If the customer agrees to pay the training cost and affords the training time, the training will begin. The customer will be categorized as expert and normal customer. For the expert customer, training metrics related to the model will be presented to the customer during the training process. Afterwards, the tips on the test data will be shared with both types of customers. If the customer is satisfied with the results, they will decide to continue using the product and pay a subscription fee. After the customer's trade decision, the product will start real-time trading and give the portfolio distribution every day. At the end of each day, 1% profit share will be collected from the customer on the return. The training cost, subscription fee, and profit paid by the customer will form the benefits of the system. At the same time, metrics such as profit, loss, cumulative profit will be displayed to the user with dashboards and different KPIs from day to day. The results, such as how much loss or profit was made with the decisions made by the agent every day, and how much the penalty is, will be shared with the customer through the dashboards.

The customer will be categorized as expert and normal customer. Expert customer will also be able to interact with the hyperparameters of the algorithm

In this part, we will describe the different parts of the system, such as the entities involved, the environment, any limitations or conditions to consider, the parameters used, and the objectives of the tool.

2.1 Entities

The entities in this system are the stocks considered for buying, selling, or holding. The system will use historical financial data and real-time market data to evaluate the performance of each stock. Other entities may include market indices, market

conditions, and various economic indicators. The tool will also need to consider available capital, investment horizon, and risk tolerance as entities in the system.

2.2 Environment

Environment of the system is consisting of market trends, global economic conditions, natural events, political factors, and regulatory policies. Those are all external factors that can impact the stock's performance. These external factors are unpredictable and constantly changing, making it challenging to optimize the portfolio value over time.

2.3 Constraints

There are several limitations and conditions that should be considered when making decisions about buying, selling, or holding stocks, including legal restrictions, market liquidity, investment horizon, risk tolerance, transaction cost incurred for each trade, and available capital.

2.4 Assumptions

Since the system's performance is evaluated through historical data some assumptions need to be made. Since the actions (buy/sell/hold) will be executed rapidly, we assume that stock market will not be affected by our reinforcement trading agent's actions.

Nonnegative balance: the allowed actions should not result in a negative balance.

Transition cost: exchange fees, execution fees vs. Trade value. Trade value should be greater than or equal to the sum of exchange fees and execution fees.

2.5 Parameters

In this system, the parameters may include market indices, technical indicators, market capitalizations, the model's hyperparameters, customers' available inputs. The reinforcement learning algorithm will be used to maximize its objective according to the parameters mentioned.

2.6 Objective

Objective of this project is to develop a portfolio management tool that maximizes the future value of the portfolio. The tool must take into account the constraints, entities, and parameters to generate the optimal portfolio allocation decisions. The primary goal is to achieve the highest possible return while managing risk effectively. The system

should also be able to adapt to changing market conditions and constraints, which is why reinforcement learning is an appropriate approach for this problem.

3. LITERATURE REVIEW

3.1 Portfolio Management

Portfolio management is a field about increasing the value of the savings and providing to meet with requirements of an individual (Stewart, Piros and Heisler, 2019). The prevalent perspective on portfolio management argues that it is a method of making logical decisions (Martinsuo, 2013). According to Kumar et al. (2008) portfolio management is usually related to financial assets. In addition to finance, NPD, research and development, and MIS have all employed portfolio management ideas (Kumar et al., 2008). There is a term of asset allocation which is an issue when it comes to deciding the proportions of several asset classes, including cash, bonds, and stocks to address the financial needs. It also creates a significant component of the structure and degree of portfolio performance by controlling variability, meeting cash flow requirements, and producing revenue growth (Stewart, Piros and Heisler, 2019). A portfolio can be defined as with the collection of containing a variety of stocks, bonds, commodities, exchange rates, etc., and managing a portfolio can be seen as a trading activity (Albadvi, Chaharsooghi and Esfahanipour, 2007).

3.1.1 Financial portfolio management

In finance, portfolio management refers to the process of managing a number of asset classes (including stocks, bonds, and cash) with the goal of maximizing return for a given time period while striving to reduce risk. Numerous subclasses can be found inside each asset class. These consist of various stock (small-cap, mid-cap, international, etc.) and bond (domestic, foreign, junk, short-term, long-term, inflation adjusted, etc.) varieties. In finance, portfolio management refers to the process of managing a number of asset classes (including stocks, bonds, and cash) with the goal of maximizing return for a given time period while striving to reduce risk. Numerous subclasses can be found inside each asset class. These consist of various stock (small-cap, mid-cap, international, etc.) and bond (domestic, foreign, junk, long or short-term,

inflation adjusted, etc.) varieties. These asset types differ in terms of liquidity and risk-return characteristics. The level of relationship among assets in a portfolio affects the risk-return characteristics, which vary from those of individual equities. Financial asset owners often choose a portfolio of assets based on their risk tolerance and strategic goals (such as retirement). For rebalance purposes (ensuring that their investment mix remains to be in line with their strategic aims), they regularly trade assets. The cost of trading affects how frequently they trade. Assets with a high degree of liquidity can be traded for comparatively less money. Markets normally decide the value of every financial asset. Actual asset expenses, trading commissions, and investment management charges (for running or sustaining a portfolio) are all included in the price of these assets (Kumar et al., 2008).

3.1.2 Portfolio optimization

Portfolio optimization aims to find the optimal allocation of assets that maximizes return for a given level of risk. Mean-variance optimization, proposed by Markowitz (1952), is a widely used approach that maximizes expected return while minimizing portfolio variance. The Kelly criterion, developed by Kelly (1956), is an alternative method that maximizes long-term growth rate by betting a fixed fraction of wealth. Recent research has shown the equivalence between the Markowitz and Kelly approaches under certain assumptions, known as the Markowitz-Kelly equivalence (Thorp, 2019). Different methods can be used for portfolio optimization, but understanding the fundamentals of these approaches is crucial for effective portfolio management.

3.1.3 Stock trading

Three main phases make up stock market trading: choosing a stock, buying a quantity of shares, and ultimately selling them for a profit (Zhang, 2001). A stock trading strategy is essential to investment firms (Liu et al., 2018). Finding the best technique, meanwhile, is difficult in the intricate and fast-paced stock market (Liu et al., 2018). That's why, it is vital to choose the right moment to buy and sell (Zhang, 2001).

An integral part of stock trading is deciding the right stock and the right time, and there exists various analytical methods to ease this decision. There are two main categories under these analytical methods called technical and fundamental analysis. Technical analysis methods consider the previous movements of a stock to predict the next

movements. Moving averages, relative strength index (RSI), and moving average convergence divergence (MACD) are some significant indexes to examine the price trends and characteristics of each stock. Besides, fundamental analysis is a technique to examine the organisation's financial statements, audit reports, income statements, periodic balance sheets, dividend records, sales records, management capabilities, and competitive environment before estimating each stock's inherent worth based on expected future cash flow. They choose to purchase a stock if the current value is less than the inherent value because they believe the price will increase. In the stock market, the process of purchase goes like this: buyers specify the price they want to buy the stock, sellers indicate the price they want to sell, and if the prices match, the trade takes place (Théate and Ernst, 2021). Dealers typically carry out their orders using a central order book, which can be seen in Table 3.1 and Table 3.2, that is shared by all participants and contains all buy or sell orders for a given security in order of price and time of order receipt. The majority of exchanges prioritise orders according to the best price, and if orders are put at the equivalent price, priority is assigned to the first order to be placed. Orders are typically displayed with price and time priority. (Nuti et al., 2011). Numerous techniques are used based on either of these two techniques alone or in conjunction. Likewise, modern trading techniques have also been applied to stock market analysis, valuation of businesses, and the detection of purchase signals. Modern trading techniques include two categories: algorithmic stock trading and stock trading with machine learning (Shavandi and Khedmati, 2022).

Table 3.1 : Order book before (a).

Order Book – ABC Inc.			
Buy		Sell	
Quantity	Price	Price	Quantity
5,000	99	99	4,000
8,000	98	100	10,000
10,000	97	101	1,000
15,000	95	103	15,000

Table 3.2 : Order book before (b).

Order Book – ABC Inc.			
Buy		Sell	
Quantity	Price	Price	Quantity
1,000	99	100	10,000
8,000	98	101	1,000
10,000	97	103	15,000
15,000	95	104	3,000

3.1.3.1 Stock trading with machine learning

With the development of information technology and computer technologies over the past ten years, financial markets have become more active, worldwide, and complicated. As a result, trading by software programs and the development of algorithmic trading systems to automate one or even more trading steps have been encouraged. In addition, any algorithmic trading system is motivated by profits. Algorithmic trading systems are being used more and more by corporate investors and administrators of hedge funds, mutual funds, and pension funds. Nowadays, between 50 and 60 % of all stocks handled in the United States and Europe are handled through these systems. In 2009, high-frequency algorithmic trading made about 60% of US equities transactions. It has been a significant source of development in computing as well as analytics, particularly in grid/GPU computing and artificial intelligence. (Nuti et al., 2011). Machine learning has become an indispensable part of finance by creating decisions about stock movements, and it is able to be used for a bulk of aims, for example an individual who wants to trade might invest in commodities futures, or trade stocks and shares on stock exchanges. In addition, the recent growth of cryptocurrencies like the Bitcoin opens up new, intriguing possibilities for machine learning in stock trading (Shavandi and Khedmati, 2022). The implementation of artificial intelligence affects the structure of the portfolio management teams and, in

turn, the efficiency of the trading. Simply, a machine can be programmed to choose stocks and build portfolios for individuals, and it can be asked to decide jointly on portfolio structure (Guida, 2019).

3.2 Machine Learning

Machine learning is an advantage to cope with difficulties that cannot be solved by conventional statistical approaches, and it tries to describe, predict, and make inferences from the data (Jiang, Gradus and Rosellini, 2020). It aims to autonomously discover meaningful correlations and patterns from samples and observations rather than codifying information into computers (Bishop, 2006). Recent developments in machine learning (ML) have made it possible for intelligent systems with cognitive abilities akin to those of humans to emerge. These systems now permeate both our professional and personal lives, shaping networked activities on digital markets in every imaginable way (Shrestha et al., 2021). Artificial intelligence (AI), the term used to describe the ability of such systems to solve complex issues, is dependent on analytical models that produce predictions, rules, solutions, suggestions, or similar outputs. The first approaches at creating analytical models depended on manually created rules that explicitly programmed known correlations, processes, and methodological approach into intelligent systems (Russell & Norvig, 1995). Analytical models are becoming rapidly constructed utilising what is often referred to as ML, driven by the viability of new programming applications, the accessibility of data, and the wide accessibility of requisite computational power (Brynjolfsson & McAfee, 2017). The difficulty of explaining and formalising knowledge into a computer form is lifted by ML, which also makes it possible to create intelligent systems more quickly. The field of machine learning (ML) has made a number of noteworthy improvements in complex learning algorithms and effective pre-processing methods during the previous few decades. One of these developments was the development of artificial neural networks (ANNs) into deeper and deeper neural network topologies with better learning capabilities, or deep learning (DL) (Goodfellow et al., 2016; LeCun et al., 2015). DL already displays superhuman capability for particular applications in closed situations by outperforming human skills. (Madani et al. 2018; Silver et al. 2018). Machine learning methods enable individuals to cope with data more effectively by teaching it to machines. In some cases, even if the data is able to be observed, explaining it is not easy. When it comes to ease of interpretability,

machine learning becomes useful. The main aim of machine learning is to gain information from the data. Supervised learning, unsupervised learning, reinforcement learning, and neural networks are various algorithms used in machine learning commonly (Manesh, 2020).

3.2.1 Supervised learning

Supervised learning is a kind of machine learning duty of learning by using previous examples of independent-dependent variables. Supervised learning requires outer support, a function is deduced from the trained data including various training samples. Inputs are separated into two categories: train and test dataset. The dependent variable of the train dataset is predicted, and the test dataset is used to identify the accuracy of this prediction. Decision trees, naïve bayes, and the supportive vector machine are three subtitles of the supervised learning (Manesh, 2020). Automated decision tree algorithms discover interactions and nonlinear significant impacts. To create decision trees and an overall "forest," predictors are piled and divided (depending on values). In bootstrapped datasets, random forests construct several trees, and they produce an aggregate tree by averaging the trees to decrease the overfit. Besides, support vector machines seek to find the optimum hyperplane having the greatest distance possible between the two spatially closest places to divide the data into groups by treating each predictor as a dimension (Jiang, Gradus and Rosellini, 2020). Last but not least, naïve bayes is a basic network which depends on the estimations. When compared to more complicated learning algorithms, Bayes classifiers typically perform less accurately. However, it is found that for some specific cases, naïve bayes show better performance for extensive comparison (Osisanwo et al., 2017).

3.2.2 Unsupervised learning

In unsupervised learning, a machine takes the inputs and tries to detect a pattern from these inputs without output prediction like in supervised learning or gaining rewards like in reinforcement learning. Principal component analysis and K-means are two most common instances of unsupervised learning algorithms (Ghahramani, 2004). Principal component analysis is an orthogonal conversion which takes the potential correlated inputs and breaks the correlation. It is a kind of dimensionality-reduction method which eases the computability and speeds up the transactions. It also uses the

linear combination for clarifying the variance-covariance formation. On the other hand, K-means is used to categorise dataset by defining k clusters, and it is more logical to centre these clusters far away from themselves. After that, the next data should be linked to the closest centre, and the locations of new k centres should be calculated (Manesh, 2020).

3.2.3 Deep learning

A subset of machine learning known as "deep learning" uses numerous layers of neurons made up of intricate structures or non-linear changes to attempt to represent high-level abstractions of data (Hao et al., 2016).

According to LeCun et al. (2015), computational models with numerous processing layers can learn representations of data at various levels of abstraction thanks to deep learning. These techniques have significantly raised the bar for object identification, visual object recognition, speech recognition, and many other fields like drug discovery and genomics. By employing the method of backpropagation to suggest how a machine should modify its internal parameters that need to be modified to compute what is represented in each layer based on the representation in the layer prior to it, deep learning can uncover detailed structure in big data sets. Recurrent nets have shed light on sequential data types like text and speech, whereas deep convolutional nets have made advancements in the processing of pictures, video, speech, and audio (LeCun et al., 2015).

3.2.3.2 Neural networks

Neural network mechanism is like a simulation of our brain cells (Figure 3.1) in a computer which is linked to each other with neurons. A clear collection of programmed commands serves as the foundation for the traditional computing method. A different computing paradigm, represented by neural networks, learns how to solve a problem by studying a collection of related problems. Studies of the data processing methods in biological nerve systems, especially the structure of the brain, were the primary source of inspiration for neural networks. Getting a better knowledge of how information is processed in living organisms is, in fact, a major goal of the majority of existing studies into neural network algorithms.

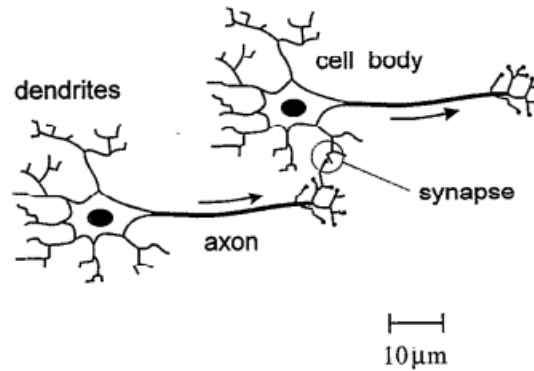


Figure 3.1 : Representation of neuron system.

The amount of neural network research has dramatically increased from the 1980s, and the topic has received substantial media coverage. While creating new algorithms and fundamental theories has occupied a large portion of research efforts, also there has been a growing focus on practical implementations. In fact, during the past several years, the topic has developed to the stage where a wide variety of practical implementations are used often in a variety of sectors. It is now evident that neural networks provide a potent collection of tools for tackling issues with some fields such as “pattern recognition, data processing, and non-linear control” that can be seen as complementing those of more traditional approaches. Particularly in the field of scientific instruments, there is a growing demand for efficient non-linear data processing approaches, and neural network approaches have a lot to offer in this context. (Bishop, 1994, p.1803).

3.2.3.1 Artificial neural networks (ANN)

In artificial neural networks modelled by McCulloch and Pitts in 1943, there are neurons like the brain which take the inputs from each other and get activated or stay inactive. Weights can be positive or negative and show the power of the neuron relations. In 1960, it was discovered by researchers that the neurons are able to apply advanced pattern recognition, in addition they can retain their performance even if some neurons are extirpated, and these basic neurons are defined as perceptrons. Neurons take inputs and the weights of these inputs are added. A specific threshold is determined, and the output turns one if the sum of these inputs is more than this threshold, else it turns to zero. On the other hand, the error back-propagation method creates complicated networks of these basic neurons learned from instances, and it

demonstrates that linearly inseparable networks are able to be solved. Back-propagation method runs to get a feed-forward mechanism which has a continuous output. Neurons are starting to learn by specifying initial weight with minimum random numbers, and inputs produce outputs. The gap between the produced output and the actual or desired output is squared, and the total of these results is defined as total error which should be minimised. The optimization of this total error can be provided by selecting the weights (Krogh, 2008).

3.2.3.2 Recurrent neural networks (RNN)

In simple neural network architecture (Figure 3.2), the network learns the network's parameters (weights) by the non-linear relationship between features and target variables that are fed into network at that very time. But, in real world, there is numerous relationship that explains target variable. RNN seeks the past features' relationship by containing hidden states spread over time (Lewis, 2016). Hidden states hold the past information of features, enabling the model to make predictions considering past information.

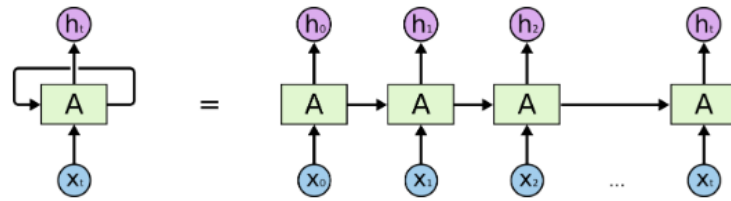


Figure 3.2 : Recurrent neural network architecture.

3.2.3.3 Long-short-term memory (LSTM)

Long-short-term memory (LSTM) is a special form of RNN that was first introduced in 1997. At that time, computational inefficiencies that RNNs have and the success of convolutional neural networks (CNNs) hindered RNNs and it became popular only after 2007. The time difference between the invention and application of RNNs is very good proof that the world of artificial intelligence does not develop on the basis of the latest research rather it develops in a retrospective way.

LSTM comes with cell-gate architecture (Figure 3.3) that deals with the vanishing gradient problems RNN suffers. Solving this problem enables the model to hold long-

term information without the interruption of irrelevant inputs (Hochreiter & Schmidhuber, 1997).

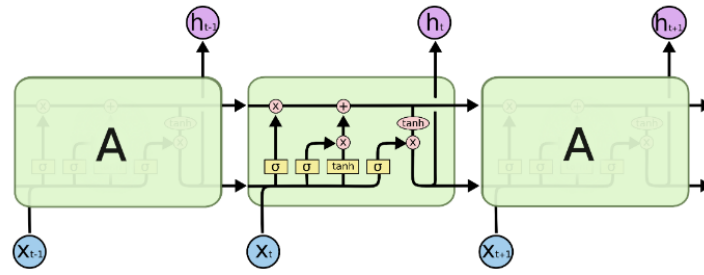


Figure 3.3 : LSTM architecture.

According to a systematic review of Li & Bastos in 2020, LSTM technique is widely applied in time series forecasting because it is the best algorithm that deals with sequential data.

3.2.4 Reinforcement learning

Reinforcement learning is a machine learning algorithm that combines two distinct methods: supervised learning and dynamic programming. The logic of reinforcement learning is basically achieving an identified goal with trials and errors (Harmon and Harmon, 1997). Reinforcement learning can be explained into two different methods: model-free and model based. While model-free models use experiences to learn, model-based models learn from the environment's known model to moderate the actions (Barfuss, 2021). Sutton and Barto (1998) explained that there is an environment in a model-based method which provides “the reward function” described as “the goal of the agent” (p.126). One of the most integral aims of the reinforcement learning is augmenting this award by exploring without any previous information. The difference and also the difficulty of this algorithm is that the reward may not be gained immediately, this kind of reward is called delayed rewards. Trial-error systems and delayed errors are two of the most significant properties of reinforcement learning. Delayed error term is also the most previous example of machine learning that is used in checker players by taking advantage of temporal-difference learning to handle these rewards (Sutton, 1992). It is an inquiry in reinforcement learning when one accumulates data to evaluate a scenario that might help him make a better choice in the future (Güçkıran, 2020). Following the exploration, individuals may pick for a specific choice. When one can make the best judgments with the maximum potential

outcome given currently acquired knowledge or information, this is referred to as exploitation in reinforcement learning. The fundamental ideas of Reinforcement Learning are exploitation and exploration, which support the agent's development of better online decision-making (Güçkıran, 2020).

3.2.4.1 Markov chains

A Markov chains (Figure 3.4) is a stochastic process that describes a sequence of events. Each state has a set of probabilities associated with it, which describe the likelihood of moving to another state. These probabilities are shown in the transition probability matrix (Figure 3.5). One of the important attributes that Markov chain has is its probability of transitioning from one state to another state depends only on the current state, not on any previous states which is also referred to as memoryless property. Markov chains have numerous applications, such as modeling stock prices, weather patterns, and natural language processing.

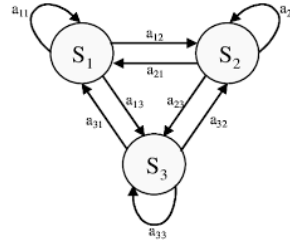


Figure 3.4 : Visual representation of markov chain with nodes and arcs.

$$P = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

Figure 3.5 : Transition probability matrix.

3.2.4.2 Markov decision processes

Markov decision processes (MDPs) are a type of Markov chain that incorporates decision-making with one another component: Actions. Actions enable control over the system. This control makes the system usable for modeling decision-making problems in which the outcomes are uncertain. In an MDP, an agent must make

decisions based on the current state of the system (s_i) to maximize some long-term reward. The agent's actions affect the system's state and the reward it receives. MDPs are widely used in reinforcement learning, optimal control, and operations research.

3.2.4.3 Reward function

Rewards determine the immediate outcome of an action. As shown in the equation 3.1, reward function is the summation of all the further rewards. Rewards can be determined systematically depending on the problem and data.

$$G_t = \sum_{k=0}^{\infty} r_{t+k+1} = r_{t+1} + r_{t+2} + \dots \quad (3.1)$$

3.2.4.4 Discount factor (γ)

In reinforcement learning domain we are interested in long-term rewards. Summing up the rewards of future states makes the quantity unbounded in stochastic processes. That's why discount factor needs to be implemented to make the equation bounded which can be calculated with geometric series (equation 3.2). Since the gamma is between 0 and 1 ($\gamma \in [0,1]$), the importance of future rewards is exponentially decreasing (Busoniu et al., 2010).

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \quad (3.2)$$

3.2.4.5 Value function

Connecting discounted sum of rewards and stochasticity of states will create the value function.

The value function estimates the long-term value of a particular state or state-action pair, based on the expected future rewards (Sutton & Barto, 2020) (equation 3.3).

$$V(s) = E[G_0 | s_0 = s] = E \left[\sum_{k=0}^{\infty} \gamma^k r_{k+1} | s_0 = s \right] \quad (3.3)$$

3.2.4.6 Policy (π)

A policy is a mapping from states to actions that tells an agent what action to take in each state. The agent selects an action based on the state it's in at a given time step.

Deterministic policy

As it seen from equation 3.4, deterministic policy outputs the action directly based on the state it's in at a given time step.

$$\pi(s) = a \quad (3.4)$$

Stochastic policy

In stochastic policy (equation 3.5), rather than mapping the states into actions directly, policy outputs a probability for each action based on the given state. Specifically, $\pi(a|s)$ represents the probability of choosing action 'a' when the agent is in state 's'.

$$\pi(s) = p(a|s) \quad (3.5)$$

3.2.4.7 Bellman equation

The Bellman equation (equation 3.6) is a fundamental concept in reinforcement learning that provides a recursive relationship between the value of a state and the value of its successor states. The equation expresses the value of a state as the sum of the immediate reward and the discounted value of the next state. It is used to compute optimal policies and value functions in MDPs and other decision-making problems.

$$V(s) = R(s) + E[\gamma V(s')] \quad (3.6)$$

3.2.4.8 Optimal policy and value iteration

The optimal policy function (equation 3.7) in reinforcement learning is the strategy that maximizes the expected long-term reward. Value iteration is an algorithm used to find the optimal policy by iteratively updating the value function for each state based on the maximum expected rewards achievable from different actions. This process

continues until the value function converges, and the resulting values determine the optimal policy.

$$\pi(s) = \operatorname{argmax}_{a \in A} \mathbb{E}_{s' \sim P(s,a)} [R(s,a) + \gamma V(s')] \quad (3.7)$$

3.2.4.9 Q-learning

Q-learning is a model-free reinforcement learning algorithm that learns the optimal policy and value function for an MDP by estimating the Q-values. Q-values represent the expected long-term reward of taking a particular action in a particular state. The algorithm updates the Q-values using the Bellman equation, which is used to compute the optimal policy and value function.

Temporal-difference learning is easing the estimation of the delayed rewards by utilizing previous and current situations (Barfuss, 2021). Temporal-difference learning has the similar logic with Q-learning which is the most popular and common algorithm in some ways. Q-learning does not figure out a map from state-to-state values, but it tries to figure out Q-values which means mapping the state-action couples to values. Rather than using the value function like in classical reinforcement learning, it has its own Q-function which includes Q-value linked with each specific action. Q-values are the total reward which is received from related action by following the policies (Harmon and Harmon, 1997).

3.2.4.10 Epsilon greedy discovery

In value function methods such as Q-Learning, the optimal policy is obtained by being greedy with Q values. However, as this situation will cause exploitation as mentioned, the agent can be made to act randomly in certain situations. Such discovery systems are called epsilon-greedy discovery algorithms. The Epsilon value takes a value from 0 to 1, with a value of 1 completely random and a value of 0 completely leads to greed. Values in between can be prepared in various ways during the training period (Güçkıran, 2020).

3.2.4.11 Function approximation

Sutton and Barto stated that function approximation is a useful technique when dealing with large or continuous state and/or action spaces, enabling the ability to make generalizations. It entails creating an approximate model of the complete function using real-world instances, commonly seen in supervised learning (2018).

3.2.4.12 Deep reinforcement learning

Deep Reinforcement Learning is an extension of Reinforcement Learning that uses deep neural networks to approximate the values or Q-values. It can handle high-dimensional state spaces and has been successful in solving complex problems, such as playing Atari games and controlling robots. Deep Reinforcement Learning is a powerful and widely used method in reinforcement learning and machine learning. In recent studies, groundbreaking achievements were made in stabilizing the learning process and attaining remarkable outcomes such as Deep Q-Network (Figure 3.6) (Mnih et al., 2015) and AlphaGo (Silver et al., 2016).

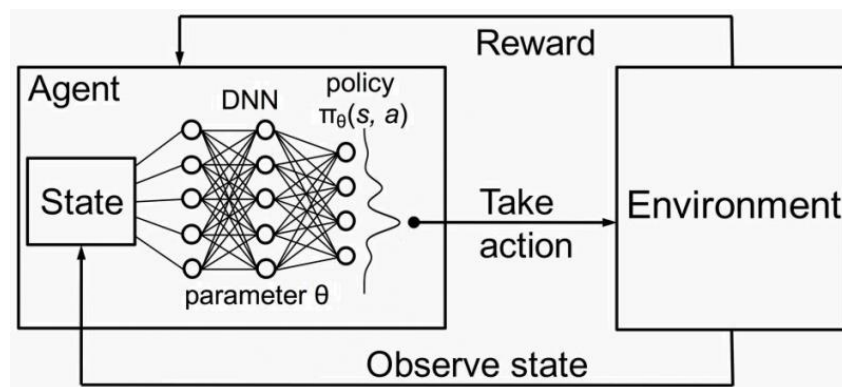


Figure 3.6 : Deep reinforcement learning architecture.

3.2.4.13 Explore method

Reinforcement Learning agents may not be able to find a better reward because they will try to reach that state again from the moment they receive a positive response from the environment, because they will exploit the reward they have found, despite the possibility that a better reward will be found. For this reason, every reinforcement learning method needs a robust exploration strategy (Güçkıran, 2020).

3.2.4.14 Exploit method

Easy adaptation to new, unexperienced or unseen environments is an important feature that reinforcement learning agents must overcome. If agents do not exploit well, they memorize the most appropriate policy in the environments seen during the training and cannot produce the desired result in the new environments. The strategies used to prevent this are called exploit methods (Güçkıran, 2020).

3.2.4.15 Data augmentation

Data augmentation is a method frequently used in cases where Deep Learning methods have difficulties in exploit methods. Data augmentation methods are also used in reinforcement learning systems, as are used in deep learning to prevent memorization. This method can be to train the agent with more environments, or it can be provided by methods such as domain randomness by intervening in the environment (Güçkıran, 2020).

3.2.4.16 Experience replay

Lin (1992) was the first to introduce experience replay. Experience replay's main principle is that train the agent using transitions that are samples from a buffer of transitions that have already been encountered. A transition is described as a quadruple (s, a, r, s_0) , where s denotes the current state, a denotes the action, r denotes the reward obtained from doing the action in the current state, and s_0 denotes the following state. The agent is trained by sampling some transitions from the replay buffer at each time step and adding the current transition to the replay buffer. It is possible to sample transitions via the replay buffer using a variety of sampling techniques, although uniform sampling is the most widely used method (Zhang & Sutton, 2017).

Experience replay has seen significant success in the deep RL field and has established itself as the industry standard in several deep RL algorithms (Zhang & Sutton, 2017).

While experience replay offers uncorrelated data for neural network training, it also significantly increases data efficiency, a desirable quality for several algorithms used in RL since they are frequently data-hungry. Although pre-deep algorithms for RL did not need to worry regarding how to balance an artificial neural network, they did need to worry about the efficiency of data (Zhang & Sutton, 2017).

Experience replay has the potential to increase efficiency of data and stable neural network training. However, the influence of several crucial transitions is delayed due to experience replay. Model deep RL systems' complexity conceals this problem. The replay buffer's size has some control over this unfavorable outcome (Zhang & Sutton, 2017).

3.2.4.17 Entropy based discovery

In policy-based or Actor-Critic methods, a probabilistic distribution is created on actions as outputs. It is possible to find the entropy of this probabilistic distribution. Since entropy is an important value for measuring how much information the system has about the environment, it is possible to use it for exploration. By trying to maximize the entropy of the system, it is aimed to learn the tasks and dynamics of the system. At the same time, during the training of the system, in cases with high entropy value, the training can be slowed down, thus directing the relevant regions to learn more (Güçkıran, 2020).

3.2.4.18 Actor-critic (A2C)

Numerous Reinforcement Learning (RL) and Neuro-Dynamic Programming (NDP) techniques fit into one of the following two classifications: Firstly, an actor-only family of methods is compatible with parameterized policies. The actor parameters are directly evaluated by simulation to determine the performance gradient, as well as the parameters are then updated in an upward manner. Such approaches may have a disadvantage due to the fact the gradient estimators potentially have a high variance. A fresh gradient is additionally computed independently of earlier estimates as the policy changes. There is therefore no "learning," as in the acquisition and consolidation of prior knowledge (Konda & Tsitsiklis, 1999).

Secondly, critic-only techniques use only value function approximation to learn an estimate of the Bellman equation's solution, which should lead to the prescription of a nearly optimal course of action. Such approaches are indirect because they don't directly attempt to maximize with a policy space. This kind of approach might be successful in creating an accurate estimation for the value function, but it lacks trustworthy assurances for the near-optimality regarding the resulting policy (Konda & Tsitsiklis, 1999).

The advantages of actor- and critic-only methodologies are combined in actor-critic methodologies. The actor's policy settings are updated in the direction of enhancement of performance by the critic using an approximation architecture (Figure 3.7) as well as simulating to develop a value function. Contrary to critic-only approaches, which ensure convergence only in very specific circumstances, such methods may have good convergence qualities if they are gradient-based. As compared to actor-only approaches, they have the potential to achieve faster convergence (owing to variance reduction) (Konda & Tsitsiklis, 1999).

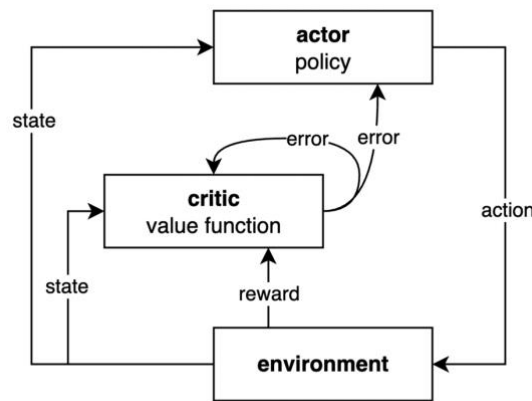


Figure 3.7 : Actor-Critic architecture.

3.2.4.19 Proximal policy optimization (PPO)

Proximal Policy Optimization (PPO) is a popular reinforcement learning algorithm that has gained significant attention in recent years. It is designed to optimize policies in reinforcement learning tasks by balancing exploration and exploitation. Several research studies have contributed to the development and improvement of PPO. Schulman et al. (2017) introduced PPO as an alternative to Trust Region Policy Optimization (TRPO) and demonstrated its superior performance and sample efficiency. They highlighted that PPO achieves better results by simplifying the optimization problem and utilizing multiple epochs of minibatch updates. Further enhancements to PPO have been proposed in subsequent studies. Wu et al. (2017) introduced a value function clipping mechanism to stabilize the learning process and prevent overly large policy updates. They showed that this modification improves the sample efficiency and convergence properties of PPO. In terms of practical

applications, PPO has been successfully applied to various domains, including robotics, game playing, and autonomous systems.

3.3 Natural Language Processing

Investment decisions have always been heavily influenced by news. It is commonly known that news about individual companies, the overall economy, and politics have a significant impact on the financial markets. The amount and density of news are both fast increasing as a result of technological advancements and increased connectivity among buyers and sellers. The use of natural language processing (NLP) in finance has become more prominent as a result of the growing amount of news data and major advances in machine learning (ML). NLP is a branch of artificial intelligence that deals with teaching computers to parse corpora of natural language in order to learn relevant information (Guida, 2019). According to Guida (2019), “Textual analysis, text mining, computational linguistics, and content analysis” are just a few of the many names that NLP goes by across many fields (p. 185). The market and investor sentiment can be significantly impacted by major news, which can cause dynamic changes in the risk characteristics of the investment universe (Mitra and Mitra, 2011).

3.4 Time Series Analysis

Nature itself contains many patterns in it. Like forms of objects, movements of planets, migration of animals. This also can be observed in human interaction with its environment. However, it is not time that causes this change, but the influence of factors that change with time. Time series analysis is concerned with the collection of observations made sequentially through time. Because of the sequential property, makes it a unique case. Contrary to most areas of statistics where observations are independent, time-series data typically do not have this property (Anderson, 1995). These phenomena are researched for a long time and impacted many different areas like business, economy, environment and etc (Tsay, 2000). Also, stock market is considered chaotic, complex, volatile and dynamic which makes prediction one of the most challenging tasks in time series forecasting (Li and Bastos, 2020).

4. METHODOLOGY

4.1 Python

Python is a stunning and powerful programming language that combines the simplicity of implementation of basic scripting as well as interpreted languages with the strength and universal application of older compiled languages. Guido van Rossum, who was working at the Netherlands' Centrum voor Wiskunde en Informatica (CWI, the National Research Institute for Mathematics and Computer Science), started developing Python in late 1989 and in early 1991, it was finally made available to the general public. Same as other programming languages, Python also has its roots in studies, where the coder struggled to complete the task at hand with the available tools and came up with and created a superior solution. Lists and dictionaries for Python are included in the language. By including these fundamental building pieces in the core language, you may promote their use, save down on project duration and code size, and produce code that is easier to comprehend. In addition, Python is known as object-oriented (OO) programming language in which data and procedure are distinct programming constructs, gain a new depth. Moreover, Python's scalable architecture and pluggable and modular design enable projects to grow while still remaining manageable (Chun, 2006).

4.1.1 Numpy

NumPy is a Python library for working with arrays. It also has the necessary functions for working in the field of linear algebra, Fourier transform and matrices. NumPy was created in 2005 by Travis Oliphant. NumPy is a Python library and is partially written in Python, but most of the fast computational parts are written in C or C++. It is an open-source project and you can use it freely. NumPy is short for Numeriacal Python (Numeric Python). In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide array objects up to 50 times faster than traditional Python lists. The array object (array) in NumPy is called "ndarray" and ndarray provides many supporting functions that make it very easy to work with. Arrays are very often used in data science where speed and resources are paramount.

4.1.2 Pandas

After collecting the data, making the data suitable for analysis is very important for data analysis. Data preprocessing and data cleaning is the phase that takes the most time from data scientists. Analyzes are made easier and faster by organizing data with the Pandas library. Pandas is one of Python's most important libraries. Pandas has methods similar to those in NumPy. While NumPy works with the same data types, Pandas can work with different data types. A data set written in Excel or a SQL table data can be easily analyzed with Pandas. Pandas is an open-source library since 2010. This library is constantly updated by developers all over the world.

Pandas has some data structures for easy analysis of data. The most used of these are the Series and DataFrame data structures. The Series data structure is one-dimensional, that is, it consists of one column. The DataFrame data structure is two-dimensional, that is, it consists of rows and columns. Everything that can be done with a dataframe in the R programming language can be done with a DataFrame in Python. The DataFrame structure in Pandas is already modeled after the dataframe structure in R.

4.1.3 Matplotlib

Matplotlib is the core plotting library of the Python programming language. It is the most widely used among Python visualization packages. Matplotlib is extremely fast at various operations. It can export visualizations to all popular formats such as PDF, SVG, JPG, PNG, BMP, and GIF. It can create line charts, scatter charts, histograms, bar charts, error charts, pie charts, box charts, and many other visualization styles. 3D plots are also possible with matplotlib. Matplotlib forms the basis of several Python libraries. Matplotlib is used to create pandas and Seaborn for example. They make it possible to access matplotlib's methods with less code.

4.1.4 Scikit-learn

Regarding moderate supervised and unsupervised tasks, Scikit-learn is a Python library that integrates a wide variety of cutting-edge machine learning algorithms. With the use of a general-purpose high-level language, this package aims to make machine learning accessible to beginners. The consistency of the API, performance, documentation, and convenience of use are prioritized. Its distribution under the

streamlined BSD license and minimum requirements promotes its application in either educational or commercial contexts (Pedregosa et al., 2011).

4.1.5 Tensorflow

According to Pang, Nijkamp and Wu (2019), Tensorflow is an adaptable library which is able to be scaled and suitable for dataflow graph-based for numerical calculations. Individuals can quickly develop, test, and apply various models from neural networks and machine learning by using Tensorflow. TensorFlow dictionary typically has two phases: the creation stage, where a graph of calculations is built, and the implementation stage, where the computational graph is performed. It is frequently used TensorFlow operations to create a computational graph which symbolizes a model of machine learning during the creation stage. There are edges and nodes in the graph. The title of the library, TensorFlow, comes from the edges, which stand in for the tensor data which will move through the graph. Operations that describe tensor calculations are the nodes in the graph. It is possible that an action has input tensors and output tensors. Simple construction pieces like fully connected layers, convolutional layers, recurrent neural network modules, and nonlinear activation factors are all provided by TensorFlow. These methods produce actions that make up the bulk of a calculation graph that carries out the calculation between input and output of the model. Additionally, TensorFlow provides a number of loss functions, including cross-entropy as well as mean square error (MSE). Among neural network functions, forward pass is completed by loss computation operations which are added on top of the output tensor. To train the model and enhance its parameters, we execute the computational graph through a series of modifying stages during the implementation stage. TensorFlow offers `tf.Session` as a method for traversing the graph and carrying out calculations. A `tf.Session` item performs actions in the graph on machines and communicates with tangible computational capabilities like local CPUs and GPUs or network nodes. With blocks, which are frequently used to generate sessions, the process is automatically closed after calculations are complete, freeing up resources. The primary method for carrying out operations and assessing tensors is `tf.Session.run()` (Pang, Nijkamp and Wu, 2019).

4.1.6 Gym

Gym is a framework that enables applying reinforcement learning methods with the pre-defined functions developed by OpenAI. It provides users with a standardized and comprehensive framework to develop and assess reinforcement learning algorithms. The module offers a diverse range of predefined environments that represent specific tasks or problems which makes applicable to all kinds of problem.

4.1.7 Yahoo finance

Yahoo Finance module provides an application programming interface (API) offered by Yahoo that allows developers to access financial data and related information. It provides a structured and standardized way to retrieve stock quotes, historical price data, company information, and other financial data, enabling developers to build applications for financial analysis and investment research.

4.1.8 Stable-Baselines3

Stable Baselines3 (SB3) is a collection of robust implementations of reinforcement learning algorithms, designed specifically for use with the PyTorch deep learning framework. It serves as the successor to the previous version, Stable Baselines.

4.1.9 FinRL

FinRL (stands for "Finance Reinforcement Learning") is a comprehensive framework designed for reinforcement learning in finance (Liu et al., 2021). It offers pre-built financial trading environments, algorithm implementations (e.g., A2C, PPO), feature engineering techniques (e.g., MACD, RSI), evaluation metrics (e.g., sharpe ratio), benchmark strategies, integration with data sources like Yahoo Finance API (4.1.4), integration with other frameworks like Gym (4.1.3), Stable-Baselines3 (4.1.5). Additionally, it provides extensive documentation and examples to facilitate understanding and implementation.

4.2 Technical Analysis

The goal of technical analysis is to forecast future price changes using asset historical prices and, occasionally, relevant trading data like transaction volume. Technical analysis focuses on identifying and exploiting price patterns of movement rather than looking at the fundamental variables influencing asset prices since it makes the

assumption that the market's price accurately reflects all pertinent information. Depending on the idea that stock prices move in trends, many widely used technical analysis approaches, technical trading systems often produce entry alerts whenever a new trend is detected and exit alerts whenever a trend finishes. On the past, they have identified trends by examining trend lines, support and resistance zones, continuing patterns, and reversal trends in a graph. Indicators are another tool used by technical traders to determine market momentum. In more recent times, more sophisticated modeling tools have been developed, such as genetic programming-based trading rules or neural network-based statistical forecasting techniques (Nuti et al., 2011).

4.3 Technical Indicators

The trading system, to put it simply, is a "collection of pre-defined" procedures that are mostly based on technical analysis indicator(s) and are used to trade call or put options. Predefined implies that trade will be well-planned for in advance and won't be random or personal. Parallel to this, the term "principles" implies that the work must be completed properly adhering to the guidelines established as rules. As a result, it is a collection of guidelines that recommend entering and leaving positions in option contracts depending on the findings of a chosen indication or indicators from technical analysis. A trading strategy enables the trading activity to be free of unpredictability (Shah, 2019).

Therefore, technical indicators will be added to RL agent's environment as state variables in further sections.

4.3.1 Sharpe ratio

The Sharpe ratio, which compares a portfolio's excess expected return to its actual volatility in return or standard deviation, is one of the most often used statistics in financial analysis (Lo, 2002).

Sharpe Ratio can be defined as $(R_p - R_f) / \sigma_f$ where R_p denotes the return of portfolio, R_f denotes the risk-free rate, and σ_f denotes the standard deviation of the portfolio's excess return (Lo, 2002).

The Sharpe ratio is now utilized in a variety of scenarios, from performance attributing to tests of market efficiency to risk management. It was initially inspired by mean-variance analysis and the Sharpe-Lintner Capital Asset Pricing Model. Expected

returns and volatilities need to be calculated in some way since these are numbers that are typically not visible (Lo, 2002).

The statistical features of returns, which can differ significantly across portfolios, strategies, and time, are what depend on whether Sharpe ratio estimators are accurate or not. In other words, the investing style of the portfolio being assessed is expected to affect the statistical features of the Sharpe ratio estimator. The reasoning behind this assertion seems straightforward on the surface: Higher volatile investment strategies have greater challenges in performance to predict than less volatile ones. It should not be a surprise that Sharpe ratios are more likely to be predicted accurately for mutual fund portfolios than for hedge fund portfolios (Lo, 2002).

The Sharpe ratio estimator's precision can be significantly impacted by the statistical traits of investment strategies like mean reversion and momentum. This is especially true when generating an annually Sharpe ratio utilizing monthly data because the investment strategy's time-series characteristics might have an impact on how predicted returns and volatility are estimated. Thus, when assessing and contrasting Sharpe ratios across various investing strategies, it is crucial to take into account the statistical nature of the return series. Sharpe ratios can be noticeably lower, especially in situations of positive serial correlation, or larger, in the case of negative serial correlation, depending on the correct multiplier, which depends on the serial correlation of the portfolio's returns. Additionally, the typical method of annualizing Sharpe ratios by multiplying monthly estimates by is correct only under very special circumstances. Sharpe ratio estimators must therefore be calculated and interpreted in the context of the specific investment style that produced the returns on a portfolio (Lo, 2002).

4.3.2 Moving average

Moving average, one of the significant indicators of stock trading, is able to be categorized as simple (equation 4.1) and exponential (equation 4.2). A basic arithmetic mean can be used to define simple moving average (SMA) due to the same logic. To illustrate, in order to find the SMA of 100 days, the closing prices of last 100 days should be added and then divided by the number of days. However, exponential moving average (EMA) gives heavier weights to latest stock prices to enable fast reaction to any price fluctuations on the recent data. This has the drawback of

occasionally being premature and causing a whipsaw due to the faster signal (Lydon, 2010).

$$\text{Simple Moving Average} = \frac{(A_1 + A_2 + \dots + A_n)}{n} \quad (4.1)$$

$$\begin{aligned} \text{EMA Today} = & \text{Price Today} \times \left(\frac{\text{Smoothing}}{1 + \text{Days}} \right) + \text{EMA} \\ & \text{Yesterday} \left(1 - \left(\frac{\text{Smoothing}}{1 + \text{Days}} \right) \right) \end{aligned} \quad (4.2)$$

4.3.3 Relative strength index (RSI)

RSI (equation 4.3) is one of the most common technical indicators for stock trading due to the high interpretability and low complexity. It is used to describe oversold and overbought by creating an index among zero and a hundred. Stocks don't typically stay in such extremes for very long due to the effectiveness of this indicator, making it a great tool for timing reversals and validating other reversal indications. When a trend reversal is seen in price, the RSI is helpful as a confirming indication. It can also work as a main determinant when momentum moves ahead of price reaction. But RSI could also produce erroneous indications, thus it needs to be verified before making new transactions. A two-part calculation is involved (equation 4.4 and equation 4.5). Initially, 14 periods are used to average the number of intervals that gained value and those that lost value (Thomsett, 2019).

$$RS = \text{Average gain} \div \text{Average loss} \quad (4.3)$$

Daily basis, the averages are modified, removing the oldest session and including the most recent one (Thomsett, 2019).

$$\text{Average gain} = (((\text{previous average}) \times 13) + \text{current gain}) \div 14 \quad (4.4)$$

$$\text{Average loss} = (((\text{previous average}) \times 13) + \text{current loss}) \div 14 \quad (4.5)$$

After the relative strength (RS) is updated, it is implemented to calculate the RSI (Thomsett, 2019). Final formula to reach the RSI given in equation (4.6).

$$\text{RSI} = 100 - (100 \div (1 + \text{RS})) \quad (4.6)$$

4.3.4 Commodity index channel (CCI)

Donald Lambert created the CCI in 1980 as a more basic technical indicator, primarily for spotting commodities' cyclical trends. Yet, as a result of its efficacy in identifying trends, it is commonly employed in the equities and Foreign Exchange (FX) markets in addition to the commodities market. It primarily indicates the trend and momentum of the stock price, which is highly volatile. It frequently provides a head start on trend reversals as well (Shah, 2019). CCI is calculated with equation 4.7 where equations 4.8, 4.9, 4.10, 4.11, and 4.12, and 4.13 holds.

$$\text{CCI} = (\text{TP} - \text{N Period SMA of TP}) - (0.015 \times \text{MD}) \quad (4.7)$$

$$\text{TP} = (\text{High} + \text{Low} + \text{Close Prices})/3 \quad (4.8)$$

$$\text{N} = \text{Look Back Period} \quad (4.9)$$

$$\text{CCI} = (\text{TP} - \text{N Period SMA of TP}) - (0.015 \times \text{MD}) \quad (4.10)$$

$$\text{SMA} = \text{Simple Moving Average} \quad (4.11)$$

$$\text{MD} = \text{Mean Deviation of TP} \quad (4.12)$$

It must be highlighted that CCI is frequently computed over a look-back period of 20 days. In the securities under examination, a value greater than 100 denotes an overbought position, while a score less than -100 denotes an oversold position. When a price is towards the top of a 20-day range, it is said to be overbought, and when it is near the bottom, it is said to be oversold. Depending on whether the value is still rising or falling, overbought or oversold trends may last for a while. According to this indicator, there are various approaches to create a choice to buy or sell a stock. According to conventional wisdom, a buy alert is produced when CCI rises above -100 (Figure 4.1). Similar to this, a sell alert is sent when the CCI drops below 100 (Shah, 2019).



Figure 4.1 : CCI in a stock chart.

4.3.5 Moving average convergence/divergence (MACD)

The MACD (Moving Average Convergence/Divergence) is a trend indicator that demonstrates the relationship between prices and moving averages. Gerald Appel

first presented the MACD in the 1970s. Between exponential moving averages for 26 and 12 days, there is a difference. The "trigger" or "signal" plot, which is a 9-day exponential average, is plotted on top of the MACD to show long/short opportunities. Calculating the MACD is as easy as comparing the exponential moving averages for 26 and 12 days. The "trigger" or "signal" plot, which is a 9-day exponential average, is plotted on top of the MACD to show long/short possibilities (Appel, 2008).

4.3.6 Directional movement index (DX)

The directional movement index (DMI) is an indicator developed by J. Welles Wilder in 1978 that identifies in which direction the price of an asset is moving. The calculations of index displayed from equation 4.13 to 4.19. The indicator does this by comparing prior highs and lows and drawing two lines: a positive directional movement line (+DI) (equation 4.13) and a negative directional movement line (-DI) (equation 4.14). An optional third line, called the average directional index (ADX) (equation 4.15), can also be used to gauge the strength of the uptrend or downtrend. When +DI is above -DI, there is more upward pressure than downward pressure in the price. Conversely, if -DI is above +DI, then there is more downward pressure on the price. This indicator may help traders assess the trend direction. Crossovers between the lines are also sometimes used as trade signals to buy or sell.

$$+DI = \left(\frac{\text{Smoothed } +DM}{ATR} \right) \times 100 \quad (4.13)$$

$$-DI = \left(\frac{\text{Smoothed } -DM}{ATR} \right) \times 100 \quad (4.14)$$

$$DX = \left(\frac{|+DI - -DI|}{|+DI + -DI|} \right) \times 100 \quad (4.15)$$

$$+DM \text{ (Directional Movement)} = \text{Current High} - PH \quad (4.16)$$

$$\text{Smoothed } +/-\text{DM} = \sum_{t=1}^{14} \text{DM} - \left(\frac{\sum_{t=1}^{14} \text{DM}}{14} \right) + \text{CDM} \quad (4.17)$$

$$\text{CDM} = \text{Current DM} \quad (4.18)$$

$$\text{ATR} = \text{Average True Range} \quad (4.19)$$

4.3.7 Larry williams R%

Relying on oscillator principles, Larry Williams %R (equation 4.14) serves as a momentum measure. This indication displays the present closing value in relation to the best high and the worst low during a specified range of time (de Almeida et al., 2016).

$$\%R = \frac{\text{Highest high} - \text{Close price}}{\text{Highest high} - \text{Lowest low}} * -100 \quad (4.20)$$

The value of Larry Williams %R is among -100 to 0. Scores under -80 are regarded as oversold, while scores over -20 are regarded as overbought for the asset. Buy alerts are generated if any oversold level of %R passes the oversold bar upward, and sell alerts are generated if any overbought level of %R exceeds the overbought line downward (de Almeida et al., 2016).

4.4 Evaluation

In reinforcement learning, there are no correct answers as in supervised learning. Due to that, assessing the performance of RL agent requires different approaches. We establish a set of comprehensive metrics that capture both the profitability and risk management aspects of trading. These metrics include measures such as total portfolio value, cumulative returns, Sharpe ratio, maximum drawdown, and other commonly used indicators in the finance industry. By analyzing these metrics, we can assess the effectiveness of our algorithm in generating consistent returns while mitigating downside risks.

4.4.1 Splitting

The train-test split is a key idea in machine learning related to model evaluation. It involves separating the available dataset into the training set and the test set. This division's goal is to evaluate a machine learning model's ability to generalize to new data. The model is trained using the training set, which enables it to learn patterns and relationships in the data. The test set, on the other hand, acts as an out-of-sample, unseen data that the model will encounter in real-world applications. Figure 4.3 illustrates the date intervals for both the training and test data.

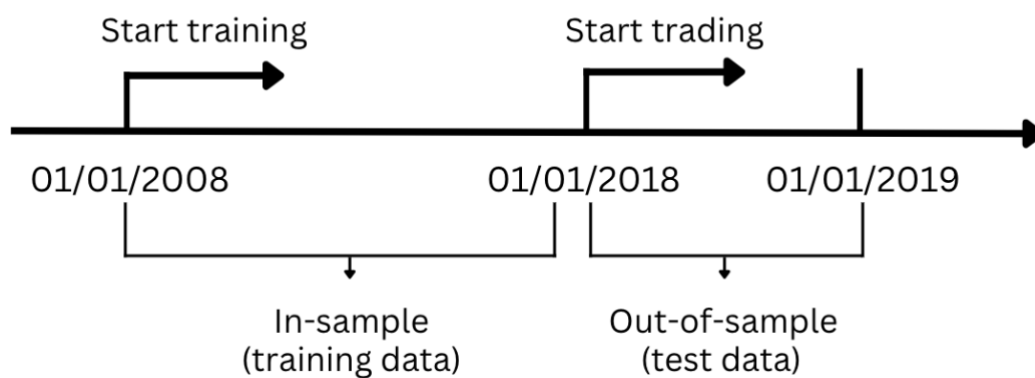


Figure 4.2 : Different sets of data.

4.4.2 Approximation error

It is stated in 3.2.4.11 that because of the large state and action spaces values need to be approximated. In Proximal Policy Optimization (PPO), the approximation error refers to the difference between the true objective function and the approximated objective function used in the algorithm.

4.4.3 Benchmarking

Creating benchmarks with different trading strategies enables comparisons between RL agent and other common strategies. Two benchmarks will be used in addition to RL agent for comparison: Random trader, Mean-variance trader. “Random trader” allocates capital to assets randomly each day. “Mean-variance trader” allocates capital to assets by the output of “mean-variance optimization” mentioned in. If the model is consistently outperforming or more stable than the two traders, we know that the model is learning from its past experiences.

5. APPLICATION

In this section, we discuss how we put our reinforcement learning agent for trading multiple stocks into practice. We provide detailed explanations of the main components using tables and figures that include pieces of code, data, and helpful graphs. Sections 5.1 to 5.6 mainly cover the technical aspects of the trading agent's backend with a real trading example. In section 5.7, we explain the user interface where the user will interact with the program.

5.1 Modules & Functions

All the necessary functions from modules introduced in 4.1 Python are imported to the kernel (Figure 5.1).

```
## import libraries

# general utils
import numpy as np
import pandas as pd
import matplotlib

# finrl
from finrl.meta.preprocessor.yahoodownloader import YahooDownloader
from finrl.meta.preprocessor.preprocessors import FeatureEngineer, data_split
from finrl.meta.env_portfolio_allocation.env_portfolio import StockPortfolioEnv
from finrl.agents.stablebaselines3.models import DRLAgent
from finrl.plot import backtest_stats, backtest_plot, get_daily_return

# gym
import gym

# tensorflow
import tensorflow as tf
```

Figure 5.1 : Importing necessary functions from modules.

5.2 Data

5.2.1 Stock selection

Stock selection relies on choosing stocks that exhibit similar characteristics such as expected return and volatility. When the stocks in a portfolio possess distinct characteristics, the RL agent tends to engage in fewer trades. With this methodology in mind, the stocks selected for inclusion are listed (Table 5.1).

Table 5.1 : Stocks included in portfolio.

	Company	Ticker
0	Apple	AAPL
1	Netflix	NFLX
2	Google	GOOG
3	Oracle	ORCL

4	Microsoft	MSFT
5	Amazon	AMZN
6	IBM	IBM

5.2.2 Data extraction

Data is extracted via “YahooDownloader” function (Figure 5.2). Data consists of daily stock prices of the stocks included in portfolio (Table 5.2).

The need for processing capacity grows with the increase in data size when training an RL agent. Due to processing limitations, we selected a decade's worth of data for training, specifically from 2008.01.01 to 2017.12.31 chosen as training data. For testing, data from 2018.01.01 to 2018.12.31 was selected, covering a period of one year. We consciously avoided incorporating more recent years to eliminate the significant financial disruptions caused by the pandemic.

However, in the interface user will be enable to change timelines.

```
# LOAD DATA

TICKERS = ["AAPL", "NFLX", "GOOG", "ORCL", "MSFT", "AMZN", "IBM"]
TRAIN_START_DATE = '2008-01-01'
TRAIN_END_DATE = '2017-12-31'
TEST_START_DATE = '2018-01-01'
TEST_END_DATE = '2018-12-31'

df = YahooDownloader(start_date = TRAIN_START_DATE,
                     end_date = TEST_END_DATE,
                     ticker_list = TICKERS).fetch_data()
```

Figure 5.2 : Code snippet demonstrating the data extraction process.

Open, high, low, close and volume information gathered (Table 5.2).

Table 5.2 : First 7 instances of data.

date	open	high	low	close	volume	tic	day
1/2/08	7.12	7.15	6.88	5.91	1079179000	AAPL	2
1/2/08	4.77	4.87	4.74	4.81	277174000	AMZN	2
1/2/08	17.26	17.37	16.88	17.07	172921700	GOOG	2
1/2/08	104.20	104.20	99.59	60.36	9940661	IBM	2
1/2/08	35.79	35.96	35.00	25.73	63004200	MSFT	2
1/2/08	3.79	3.89	3.66	3.76	12527900	NFLX	2
1/2/08	22.55	22.82	22.38	18.56	42775700	ORCL	2

5.2.3 Data split

Data is splitted into train and test sets via code snippet (Figure 5.3).

```
# 2008.01.01. - 2017.12.31
train = data_split(df, start=TRAIN_START_DATE, end=TRAIN_END_DATE)

# 2018.01.01. - 2018.12.31
test = data_split(df, start=TEST_START_DATE, end=TEST_END_DATE)
```

Figure 5.3 : Data splitting process.

5.2.4 Data analysis

Data can be analyzed through visualizations and investment metrics.

According to the Figure 5.4, NFLX has taken a leap forward in the last 4 years or so, showing a different trend. It can also be observed in the Figure 5.5, the correlation between NFLX's returns and the other stocks' returns are lower than other correlations. However, other stocks are in a more stable trend. Leaving NFLX aside, IBM has the same trend in other stocks, but its price is higher than the others.

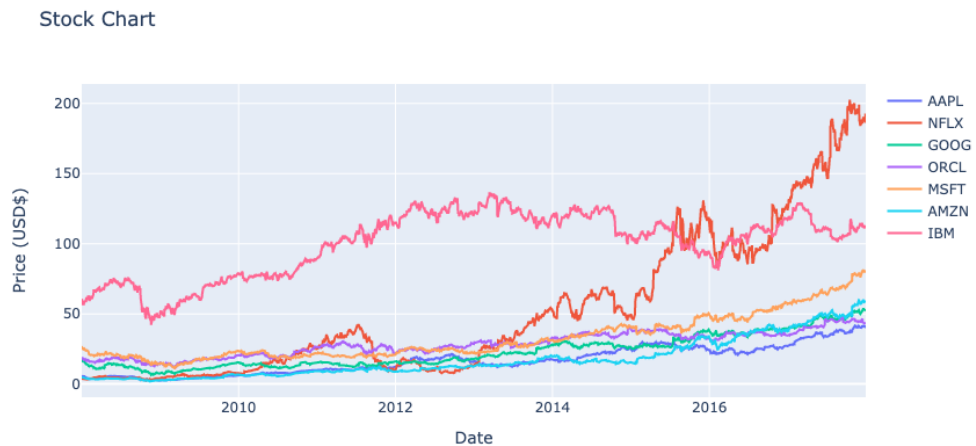


Figure 5.4 : Stock chart.

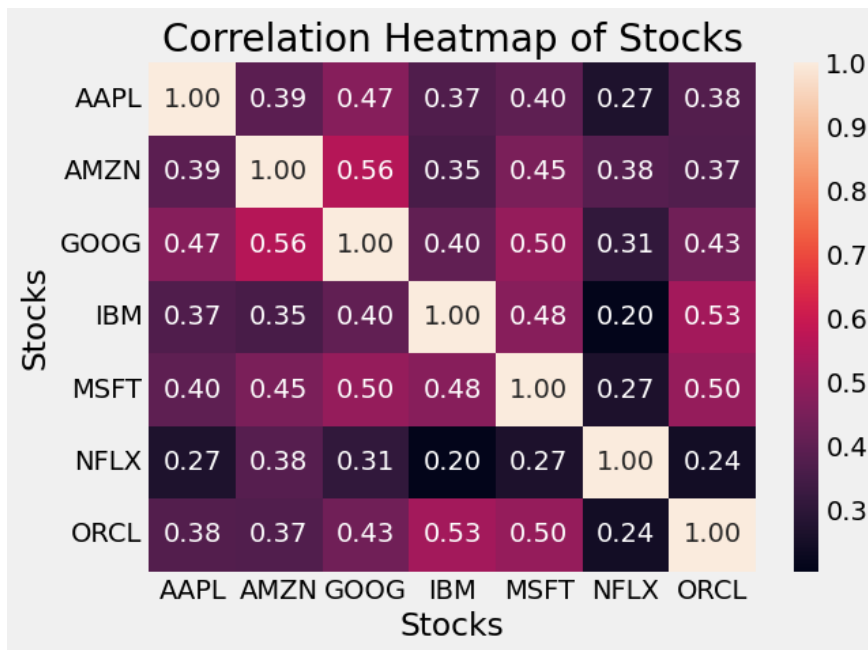


Figure 5.5 : Line plot of stocks returns between 2008-2018.

Stocks other than AAPL have positive skewness so it can be said that these stocks may have smaller negative returns and larger positive returns, and they may be considered to carry more downside risk. On the contrary, one might think that AAPL has smaller positive returns and larger negative returns and has more upside potential. Besides, all stocks have a Kurtosis value greater than 3, so returns on the whole stock may be more volatile and unusual events may be more likely to occur. In addition, a positive sharpe ratio indicates a return that exceeds the risk of the investment, but all ratios close to zero indicate that the return does not exceed the risk to a large extent. According the Table 5.3, NFLX is the highest sharpe ratio so its return is more satisfactory than others according to the risk.

Table 5.3 : Common metrics of stocks selected from 2008-2018.

	AAPL	AMZN	GOOG	IBM	MSFT	NFLX	ORCL
Anualized Returns (period:12)	0.009	0.012	0.005	0.003	0.005	0.019	0.004
Annualized Volatilities (period:12)	0.068	0.084	0.064	0.048	0.061	0.121	0.061
Skewness	-0.243	1.034	1.072	0.089	0.468	0.734	0.096
Kurtosis	10.239	15.089	18.165	9.614	14.203	22.944	10.063
Is Normal (p_value:0.05)	False	False	False	False	False	False	False
Semi Deviation	0.014	0.016	0.012	0.010	0.012	0.023	0.012
VaR Historic (percentile:0.05)	-0.119	-0.122	-0.096	-0.069	-0.108	-0.247	-0.098
VaR Gaussian (percentile:0.05, modified:False)	0.064	0.079	0.060	0.045	0.057	0.112	0.057
Sharpe Ratio (period:12, risk-free rate:0.001)	0.120	0.130	0.068	0.041	0.073	0.149	0.050

5.2.5 Adding technical indicators

As mentioned in Section 4.3, technical indicators (MACD, 30 days SMA, 60 days SMA, 30 days RSI, 30 days CCI, and 30 days DX respectively) were added via “FeatureEngineering” function (Figure 5.6).

```
# ADD INDICATORS

INDICATORS = ['macd',
               'close_30_sma',
               'close_60_sma',
               'rsi_30',
               'cci_30',
               'dx_30']

# VR
fe = FeatureEngineer(use_technical_indicator=True,
                    tech_indicator_list = INDICATORS,
                    use_turbulence=False,
                    user_defined_feature = False)

df = fe.preprocess_data(df)
```

Figure 5.6 : Adding indicators.

The RSI value is used to determine the stock's momentum and overbought/oversold conditions. According the Table 5.4, for the AAPL, AMZN, and GOOG, RSI values drop below 30, the stocks are considered to have entered the oversold territory. If the RSI falls into the oversold zone (30-), the CCI shows a negative value and the MACD line crosses the signal line to the upside, the stock may be oversold and the price may move higher.

Table 5.4 : Table consists of added technical indicators.

date	tic	macd	close_30_sma	close_60_sma	rsi_30	cci_30	dx_30
1/29/08	AAPL	-0.25	4.95	4.95	17.12	-122.70	91.48
1/29/08	AMZN	-0.10	4.13	4.13	18.69	-107.26	57.09
1/29/08	GOOG	-0.41	15.38	15.38	24.30	-120.12	65.08
1/29/08	IBM	0.47	59.02	59.02	54.88	94.11	22.90
1/29/08	MSFT	-0.21	24.57	24.57	36.06	-86.95	9.40
1/29/08	NFLX	-0.03	3.33	3.33	41.81	-14.63	25.85
1/29/08	ORCL	-0.24	17.63	17.63	36.14	-134.26	56.90

5.3 Model Design

This section presents an overview of the developed reinforcement learning agent for multi-stock trading. It includes the design of the environment, specifying the state, observation, action spaces, and the reward mechanism. The section also covers the policy and its optimization method for efficient decision making.

5.3.1 Overview

The model, built on PPO, serves as the core component of our algorithm, enabling it to learn and improve its trading policies through iterative interactions with the environment. Figure 5.7 shows the architecture of model to gain understand how the components are related to each other. Components will be explained more deeply in subsequent sections.

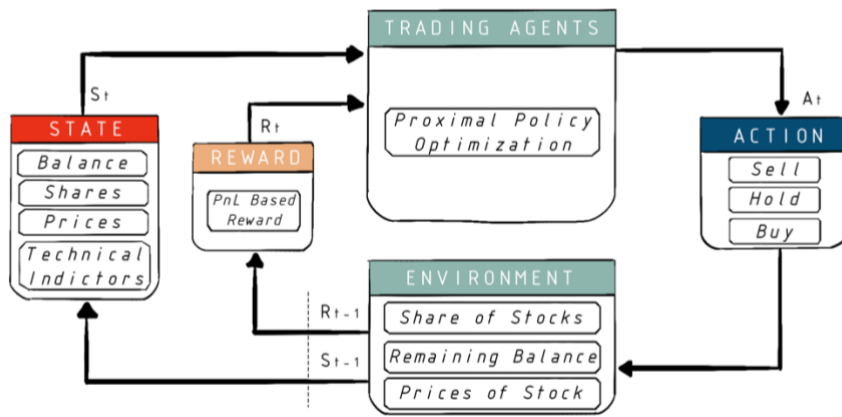


Figure 5.7 : Architecture of PPO method.

5.3.2 Environment design

Environment refers to the combination of internal and external systems in which an RL agent interacts, gets rewards, and updates its policies. Environment’s components are described in this part.

A custom class called “StockPrtfolioEnv” which is inherited from “gym.Env” class is created (Figure 5.8). This class includes all the necessary components of the environment like state space, trading agents, rewards, and actions as attributes of “StockPrtfolioEnv” class (Figure 5.9).

```
class StockPortfolioEnv(gym.Env):
```

Figure 5.8 : Class that defines environment.

```
def __init__(self,
            df,
            stock_dim,
            initial_amount,
            state_space,
            action_space,
            tech_indicator_list,
            day = 0,
            init_w = [.095,.19,.05, .095,.19,.095,.19,.095]):
```

Figure 5.9 : Attributes defined in environment class.

5.3.2.1 Observation space

The observation space represents the current situation or configuration of the environment. It includes all the important information that the agent needs to make decisions which consist of shares, prices, and technical indicators.

Dimension of state space is represented in equation 5.1 to have better understanding of the state space components. Also, “observation_space” is defined as “spaces.Box” object (Figure 5.10) which take any value from negative infinity to positive infinity.

$$\binom{Dimension}{of} \binom{state space}{state space} = \binom{Current}{balance} + \binom{Shares and}{values of each} \binom{stock}{stock} + \binom{Technical}{indicators} \binom{of each stock}{of each stock} \quad (5.1)$$

```
self.observation_space = spaces.Box(low=-np.inf, high=np.inf,
                                   shape = (1 + 2*self.stock_dim +
                                             len(self.tech_indicator_list)*self.stock_dim,))
```

Figure 5.10 : Defining observation space attribute.

5.3.2.2 Action space

Action space is also defined as “spaces.Box” object (Figure 5.11) which can take any value between 0 and 1. It corresponds to the allocation on each trading day. Thus, the total allocation units of each stock will be summed up to 1 (100%). Dimension of action space is equal to the stock dimension which is 7.


```
self.action_space = spaces.Box(low = 0,
                                high = 1,
                                shape = (self.stock_dim,)
                                )
```

Figure 5.11 : Defining action space.

5.3.2.3 Reward function

The reward (3.2.4.3) is the motivation factor of the agent to maximize its returns. Defining the right reward function is very important because the agent will act on maximizing that function which has a great influence on whole optimization process. There are various reward functions for stock trading environment and each has pros and cons.

PnL (profit and loss) based return

We defined the objective function as PnL based return function. It corresponds to a pure return function. Also, a penalty parameter was added which corresponds to transaction costs and fees. In real life, transaction costs differ on a daily basis. But in terms of simplicity, the penalty is assumed to be 1% of the allocated total values of the shares.

Equation 5.2 calculates the difference in allocations of each stock between two consecutive time periods (i and i-1) as a vector. It takes the absolute value of the subtraction of the allocation vector at time period i from the allocation vector at time period i-1. Ends up providing the magnitude of the change in allocations as a vector.

$$\left(\text{Allocation difference} \right)_i = \left| \left(\text{Allocations of stocks} \right)_i - \left(\text{Allocations of stocks} \right)_{i-1} \right| \quad (5.2)$$

Equation 5.3 calculates penalty for the current time period i (denoted by Penalty_i). It is determined by dot producting the allocation difference vector (from Equation 1) by the stock value vector at time period i (denoted by $\text{SV}_{i=j}$) and then multiplying the result by 1/100.

$$\text{Penalty}_i = \left(\text{Allocation difference} \right)_i \cdot \left(\text{SV}_{i=j} \right) \cdot (1/100) \quad (5.3)$$

Equation 5.4 calculates the portfolio value at time period i . It is obtained by multiplying the allocation of stocks vector at time period i by the respective stock values vector (denoted by $SV_{i=j}$).

The penalty (from equation 2) is then subtracted from this value to adjust for any deviations in the allocations.

$$PV_i = \left(\begin{matrix} \text{Allocations} \\ \text{of stocks} \end{matrix} \right)_i \cdot (SV_{i=j}) \quad (5.4)$$

Equation 5.5 calculates the reward at time period i by subtracting the portfolio value at the previous time period ($i-1$) from the current portfolio value (denoted by $PV_{i=j}$). Additionally, the penalty (from equation 2) at time period i is subtracted from this difference. This equation provides a measure of the change in portfolio value, taking into account both the previous value and any penalties incurred.

$$Reward_i = PV_i - PV_{i-1} - Penalty_i \quad (5.5)$$

Using PnL-based return function can potentially decrease the number of trades and will push the agent to long-term investing rather than trading. However, since our portfolio consists of only 7 stocks that have similar characteristics (5.2.4) it did not occur in our experiments. Figure 5.18 which will be explained in further section demonstrates that actions are allocated evenly between each stock.

As shown in Figure 5.11, reward function is defined in “StockPortfolioEnv” class.

```
# calculate portfolio return
alloc_diff = abs(self.actions_memory[-1] - self.actions_memory[-2])
penalty = np.dot(alloc_diff * self.data.close.values[-2])*(1/100)
value_t = np.dot(self.actions_memory[-1], self.data.close.values[-1])
value_t_1 = np.dot(self.actions_memory[-2], self.data.close.values[-2])
portfolio_return = value_t - (value_t_1 + penalty)
# save return
self.portfolio_return_memory.append(portfolio_return)
```

Figure 5.12 : Reward function.

5.3.2.4 Initialising

Before model training, initialising need to be set for the environment's and RL agent's parameters.

Environment

The investment amount is selected as 1000000 \$ (one million dollars). Necessary parameters to initialise environment are given as keyword arguments (Figure 5.13) which are explained in previous sections.

```
# initialise environment arguments
env_kwargs = {
    "initial_amount": 1000000,
    "state_space": state_space,
    "stock_dim": stock_dimension,
    "tech_indicator_list": tech_indicator_list,
    "action_space": stock_dimension,
}
```

Figure 5.13 : Initializing environment's parameters.

Agent

As shown in Figure 5.14 hyper-parameters about agent are set. These parameters setting an example. Expert users will enable to alter the hyperparameters.

```
# initialise FinRL agent class with train environment
agent = DRLAgent(env = env_train)

# initialise model arg
PPO_PARAMS = {
    "n_steps": 20_480,
    "ent_coef": .01,
    "learning_rate": .001,
    "batch_size": 5_120,
    "gamma": .99,
    "gae_lambda": .95,
    "clip_range": 0.3,
    "vf_coef": .5,
}

# initialise train model
model_ppo = agent.get_model("ppo", model_kwargs = PPO_PARAMS)
```

Figure 5.14 : Initialising RL agent's parameters.

5.3.2.5 Policy optimization method

PPO which is one of the model-free reinforcement algorithms is used as a policy optimization method.

5.4 Model Training

Then, model started training with execution of the code snippet (Figure 5.15).

```
trained_ppo = agent.train_model(model=model_ppo,  
                                tb_log_name='ppo',  
                                total_timesteps=10_240_000)  
✓ 316m 59.5s
```

Figure 5.15 : Training of RL agent.

The training process required approximately six hours and seventeen minutes to complete. It depends on various factors like time complexity of the algorithm, the size of the data, hyperparameters, and the computer system is used to train.

5.5 Model Evaluation During Training

Throughout the training period, the function (Figure 5.15) generated summaries at every iteration and batch, allowing for the ongoing evaluation of the training agent's performance.

It displays beginning total values, end total value, and sharpe ratio (4.3.1) of portfolio after each trade (Figure 5.16).

```
=====
begin_total_asset:1000000      begin_total_asset:1000000
end_total_asset:5809728.23440952 end_total_asset:1689643205.2130663
Sharpe: 1.1835394774458117     Sharpe: 4.1968037672963225
=====
```

Figure 5.16 : Performance metric at iteration (trade) 528 and 18293.

Additionally, in every batch model approximation metrics are displayed in Figure 5.17 and explained respectively.

time/		time/		time/	
fps	515	fps	511	fps	538
iterations	2	iterations	40	iterations	500
time_elapsed	79	time_elapsed	1603	time_elapsed	19018
total_timesteps	40960	total_timesteps	819200	total_timesteps	10240000
train/		train/		train/	
approx_kl	0.015743759	approx_kl	0.032777637	approx_kl	0.28260934
clip_fraction	0.0509	clip_fraction	0.132	clip_fraction	0.294
clip_range	0.3	clip_range	0.3	clip_range	0.3
entropy_loss	-11.4	entropy_loss	-12.4	entropy_loss	-16.1
explained_variance	-6.79	explained_variance	0.387	explained_variance	0.996
learning_rate	0.001	learning_rate	0.001	learning_rate	0.001
loss	-0.126	loss	-0.156	loss	-0.179
n_updates	10	n_updates	390	n_updates	4990
policy_gradient_loss	-0.0132	policy_gradient_loss	-0.0182	policy_gradient_loss	-0.00396
reward	-0.0012618081	reward	-0.00024234525	reward	0.0019194294
std	1.01	std	1.14	std	1.95
value_loss	0.042	value_loss	0.000181	value_loss	2.42e-05

Figure 5.17 : Performance metrics at batch 2, 40, and 500 (final batch).

5.5.1 KL divergence approximation

It is a metric important for training stability, measures the divergence between updated and previous policies, seeking low, stable values below 0.01 to prevent large policy updates.

5.5.2 Clip fraction

The clip fraction gauges the percentage of actions clipped during policy updates, aiming for a moderate proportion around 0.2 to 0.3 to control policy changes.

5.5.3 Clip range

The clip range determines the maximum allowed difference between updated and old policies, with its optimal range varying based on the specific problem and agent dynamics.

5.5.4 Entropy loss

Entropy loss penalizes actions with high certainty to promote diverse actions and exploration, with its optimal range depending on the problem and desired exploration level.

5.5.5 Explained variance

Explained variance evaluates the value function's prediction quality of the rewards, favoring a higher correlation between predicted and actual rewards. It is seen that explained variance increased from -11.4 to 0.996.

5.5.6 Learning rate

The learning rate, crucial for the speed and stability of the learning process, controls the update step size, with an optimal value chosen based on experimentation, often around $1e-3$ or $3e-4$.

5.5.7 Loss

Loss represents the total loss function in training, combining policy gradient loss and value function loss, and is typically minimized.

5.5.8 Number of updates

Number of updates refer to the number of updates performed during training, with no specific optimal range as it's a reflection of training progression.

5.5.9 Policy gradient loss

Policy gradient loss, quantifying policy adjustments, aims to be minimized.

5.5.10 Reward

Reward, a performance indicator, aims to be maximized in long term.

5.5.11 Standard deviation (std)

The standard deviation measures the spread or variability of values within a metric, with a lower value indicating more consistent and reliable results.

5.5.12 Value loss

Lastly, value loss assesses the discrepancy between predicted and actual rewards, helping gauge the quality of the value function, and typically aims to be minimized during training.

However, it is important to note that being able to correctly approximate the values does not necessarily mean that the RL agent is successful in trading. It means that the RL agent learned how to associate the states with rewards. If states do not have enough predictive power, it will be unsuccessful either way.

Also, the optimal ranges provided are general guidelines and can vary depending on the specific problem, environment, and agent configuration. It's important to

experiment and tune these parameters based on your specific requirements and domain expertise.

5.6 Testing / Evaluation

5.6.1 Actions

Action table (Table 5.5) visualized with box plot (Figure 5.18) to assess the stability of actions made in test data.

Table 5.5 : Allocations of each stock through time.

	AAPL	AMZN	GOOG	IBM	MSFT	NFLX	ORCL
date							
2018-01-02	0.000	0.000	0.000	0.000	0.000	0.000	0.000
2018-01-03	0.202	0.202	0.125	0.108	0.202	0.081	0.081
2018-01-04	0.099	0.248	0.099	0.099	0.248	0.109	0.099
2018-01-05	0.209	0.093	0.209	0.093	0.209	0.093	0.093
2018-01-08	0.217	0.087	0.217	0.087	0.217	0.087	0.087
...	...	...	...	...	...	...	...
2018-12-24	0.097	0.244	0.097	0.097	0.244	0.110	0.111
2018-12-26	0.192	0.192	0.077	0.077	0.192	0.192	0.077
2018-12-27	0.216	0.097	0.097	0.181	0.097	0.097	0.216
2018-12-28	0.292	0.117	0.117	0.117	0.117	0.117	0.123
2018-12-31	0.229	0.213	0.091	0.091	0.091	0.091	0.192

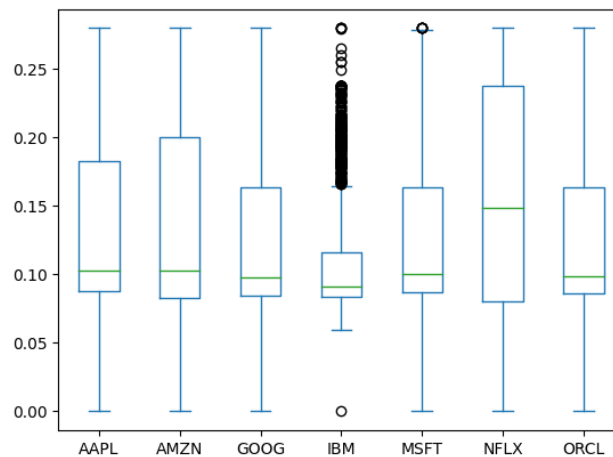


Figure 5.18 : Box plot of allocations.

The actions appear to be stable, indicating that the allocations largely remain consistent, with the exception of NFLX. Mean of the allocation on NFLX is largely differs from other allocation averages. This action of the RL agent is expected and can be explained by the upward trend of the NFLX from 2008 to 2019 (Section 5.2.4).

Furthermore, each stock has a distinct range, showing that the spread out and diversified. In other words, the allocations are not focused on a single point.

Also, as shown in Figure 5.19, allocations in each day can be observed.

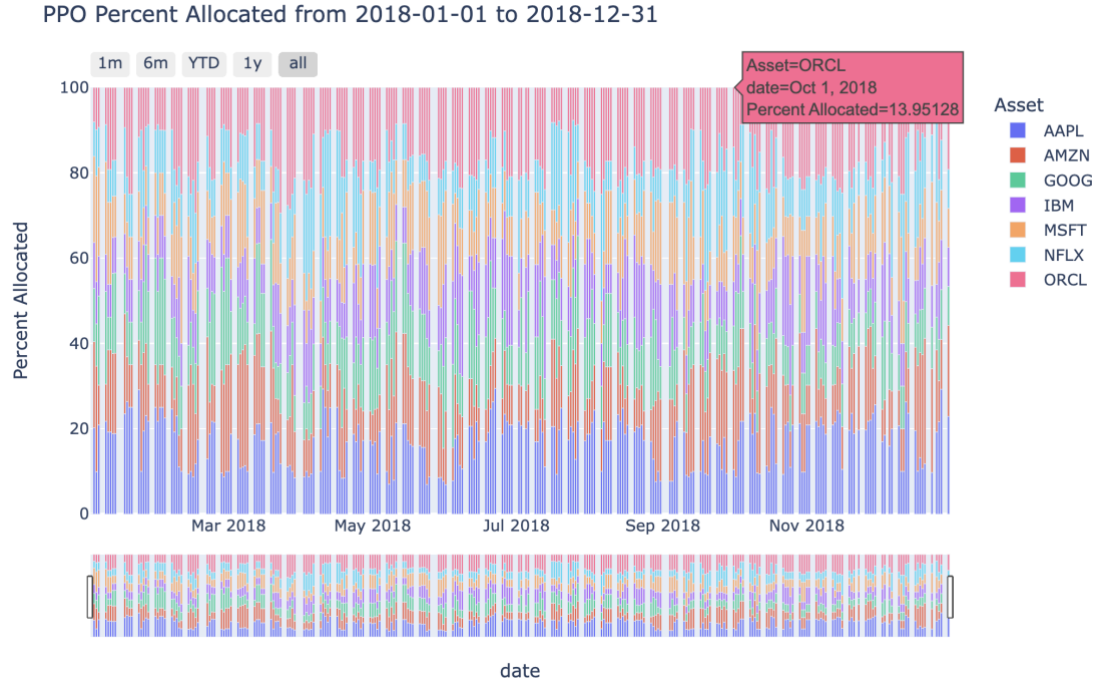


Figure 5.19 : Graphical representation of allocations through time.

5.6.2 Comparison between benchmarks

There is one agent that allocates its resources by trying to minimize variance, while there are six other random agents that randomly decide how to allocate their resources using different random seeds. The results of the daily returns for each allocation method are showed (Table 5.6). The code used to create the table is also displayed (Figure 5.20).

Table 5.6 : Benchmark results.

	PPO	Min Var	Random1	Random2	Random3	Random4	Random5	Random6
Annual return	9.859875	-7.747060	0.122289	-0.665504	1.551515	0.360295	-0.853797	-3.541403
Cumulative returns	9.818888	-7.717535	0.121804	-0.662872	1.545310	0.358863	-0.850424	-3.527600
Annual volatility	23.150632	21.406053	22.343327	22.642929	22.429736	23.266441	23.049205	22.063333
Sharpe ratio	0.521509	-0.269640	0.117037	0.083521	0.180584	0.131587	0.077807	-0.053237
Calmar ratio	0.434870	-0.293457	0.004856	-0.023389	0.059047	0.015684	-0.034796	-0.135475
Stability	0.304973	0.000012	0.009639	0.001743	0.083842	0.035286	0.000047	0.008899
Max drawdown	-22.673132	-26.399265	-25.182759	-28.453233	-26.275825	-22.972620	-24.537159	-26.140696
Omega ratio	1.097366	0.954684	1.020731	1.015056	1.032379	1.023658	1.013900	0.990696
Sortino ratio	0.742621	-0.357287	0.158926	0.113777	0.247528	0.179683	0.106134	-0.071810
Skew	0.047186	-0.349714	-0.250787	-0.185614	-0.203753	-0.192434	-0.156749	-0.243057
Kurtosis	2.420572	0.980500	1.699641	2.504815	1.341037	2.085905	2.107508	1.680285
Tail ratio	0.848266	0.721858	0.759256	0.778049	0.797419	0.792975	0.764499	0.704118
Daily value at risk	-2.868796	-2.719814	-2.804618	-2.845236	-2.809808	-2.919147	-2.896810	-2.784380
Alpha	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
Beta	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

```

# Make dataframe of stats
dict_strat = {'PPO' : DRL_strat,
              'Min Var' : minv_strat,
              'Random1' : rand_strat_1,
              'Random2' : rand_strat_2,
              'Random3' : rand_strat_3,
              'Random4' : rand_strat_4,
              'Random5' : rand_strat_5,
              'Random6' : rand_strat_6,
              }

# initialise pyfolio
perf_func = timeseries.perf_stats
# create dictionary with stats
d_stats = {}
for name, strat in dict_strat.items():
    # get strategy stats
    d_stats[name] = perf_func(returns=strat,
                             factor_returns=strat,
                             positions=None,
                             transactions=None,
                             turnover_denom="AGB")
df_stats = pd.DataFrame(d_stats)

```

Figure 5.20 : Creating summary metrics by "timeseries.perf_stats" function.

Breaking down the meaning and importance of each metric, and how PPO agent performed:

5.6.2.1 Annual return

The PPO agent's annual return is significantly higher than all other agents. This metric shows the annualized expected return over the returns of testing period, indicating that the PPO agent is generating much higher profits on an annual basis.

5.6.2.2 Cumulative return

Similar to annual return, cumulative return is a measure of the total return of a portfolio over the entire period. PPO agent also excels in this metric, leading to an overall larger increase in portfolio value over time.

The cumulative return is almost equal to the annualized return because the testing period is one year.

As shown in Figure 5.21, cumulative return can also be observed through time.

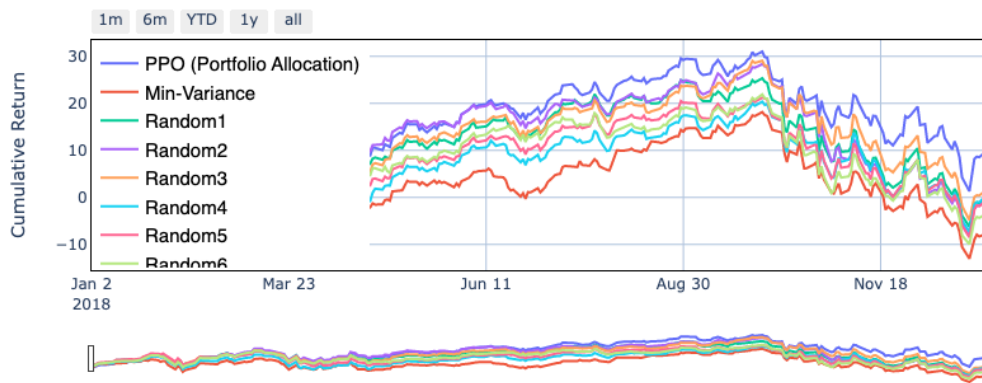


Figure 5.21 : Cumulative return graph through time.

5.6.2.3 Annual volatility

This metric represents the degree of variation of returns over a year. Despite having higher volatility than some of the other agents, the PPO agent's performance is still very competitive. This suggests that the higher return compensates for the increased risk (volatility).

5.6.2.4 Sharpe ratio

This is a measure of the performance of an investment compared to a risk-free asset, after adjusting for its risk (volatility). It is the average return earned in excess of the risk-free rate per unit of volatility. The PPO agent has the highest Sharpe ratio, indicating a better risk-adjusted return.

5.6.2.5 Calmar ratio

This ratio compares the average annual compound rate of return and the maximum drawdown risk of commodity trading advisors and hedge funds. The PPO's Calmar ratio is also the highest, implying better returns relative to the risk of large losses.

5.6.2.6 Stability

PPO leads in this metric as well, indicating a more consistent return than the other agents.

5.6.2.7 Max drawdown

This represents the largest single drop from peak to bottom in the value of a portfolio, before a new peak is achieved. PPO's Max Drawdown is competitive; while it's not the lowest, its higher return rates may justify the slightly higher drawdown.

5.6.2.8 Omega ratio

This is a risk-return performance measure of an investment asset. The PPO agent has the highest Omega ratio, showing it has a higher probability of high returns than losses.

5.6.2.9 Sortino ratio

Similar to the Sharpe ratio, this metric also measures return per unit of risk, but it only considers downside risk. The higher Sortino ratio for the PPO agent suggests that it performs well when considering only negative volatility.

5.6.2.10 Skew

Skewness is a measure of the asymmetry of the probability distribution of a real-valued random variable about its mean. The PPO's Skew is relatively close to zero, indicating a relatively symmetrical distribution of returns.

5.6.2.11 Kurtosis

This measures the "tailedness" of the probability distribution of a real-valued random variable. The PPO's Kurtosis is close to 3 (for a normal distribution), indicating a relatively balanced distribution.

5.6.2.12 Tail ratio

This is the ratio of the average of the 10% largest returns to the average of the 10% largest losses. PPO's tail ratio is not the highest but it's competitive, suggesting that it can have large gains that offset the losses.

5.6.2.13 Daily value at risk (VaR)

This measures the level of financial risk over a specific time frame. PPO's VaR is competitive, suggesting that it has a balanced level of risk.

In conclusion, the RL agent trained using the PPO algorithm outperforms the other agents in many crucial metrics, suggesting that it is more capable of maximizing returns while maintaining a balanced risk profile.

As a conclusion of the testing, we can conclude that our customized RL agent is successful in trading in multi-stock environment, capturing the upword trend.

5.7 Tool Interface

In addition to implementation, a concept interface that the potential user will interact with is created as last component of the tool. Flow chart (Figure 5.22 and Figure 5.23) describes the user's journey any how it interacts with the backend of the application.

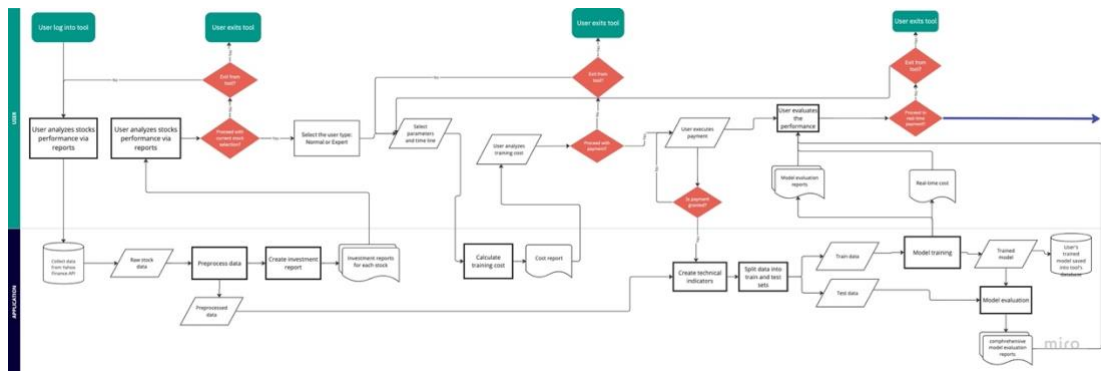


Figure 5.22 : Flow chart of tool (a).

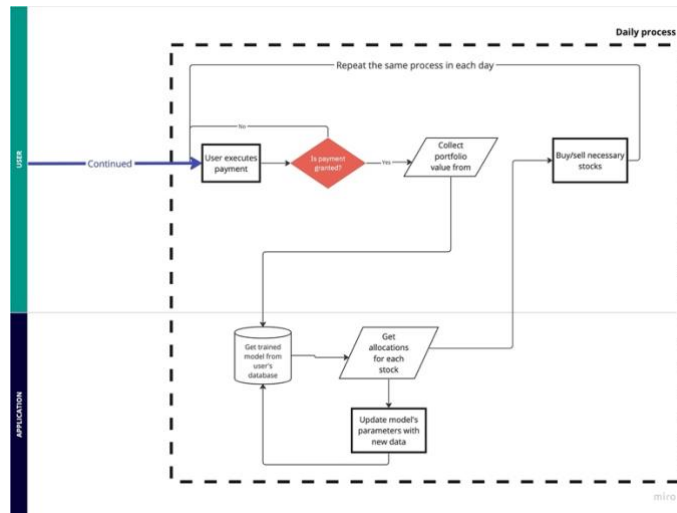


Figure 5.23 : Flow chart of tool (b).

The user's journey with the tool begins with Homepage (Figure 5.24) by logging in and selecting one of two available roles: Expert or Normal user. The expert user should have the technical knowledge about the tool, which he/she will have more flexibility with the custom functions.

Smart Portfolio
Homepage

Create new Account
Already Registered? Log in here.

NAME
Eren Kotar

EMAIL
hello@itu.edu.tr

PASSWORD

DATE OF BIRTH
Select

USER TYPE
Normal Expert

sign up

Figure 5.24 : Homepage of tool

In the Analysis page (Figure 5.25), the user specifies the stocks and timeline of interest. The tool, working in the background, connects to the Yahoo Finance API to collect the requested stock data within the specified timeline. The user then has the

opportunity to analyze the performance of the stock reports via various metrics and visualizations (**5.2.4 Data analysis**).

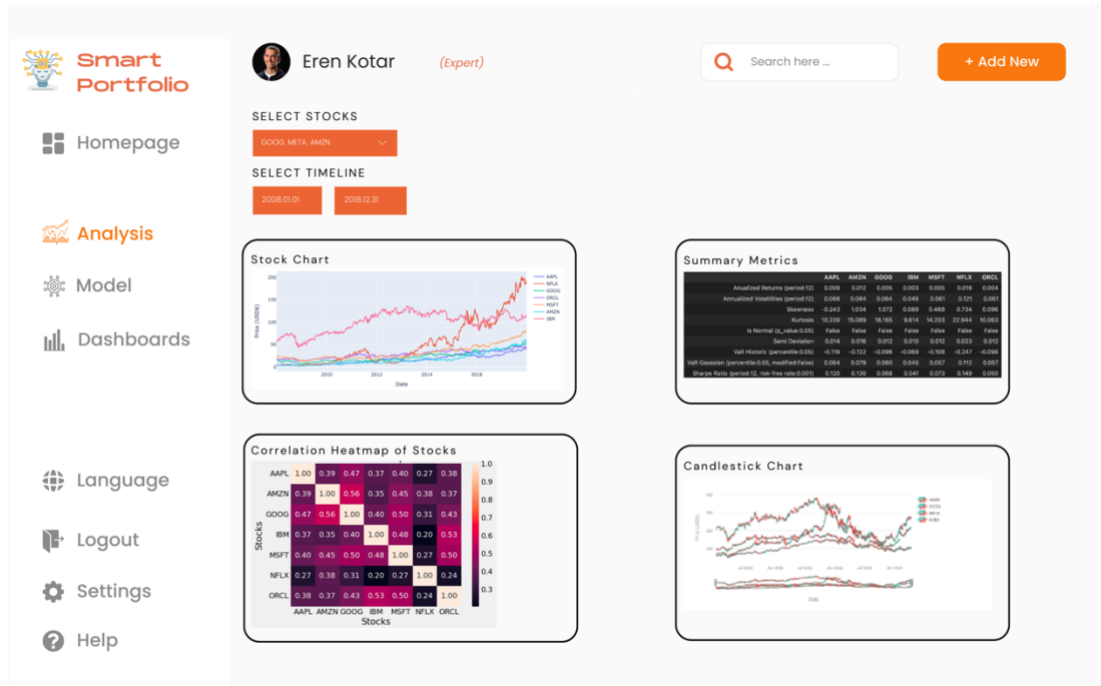


Figure 5.25 : Analysis page

If the user decides to proceed, the training phase of the trading model starts (Figure 5.26). Normal users merely select the stocks and timeline for training, while Expert users have the added ability to define the hyperparameters of the Reinforcement Learning (RL) agent. Again, in the background, the tool gathers the needed stock data, splits the data into training and test sets (**5.2.3 Data split**), and computes the training cost and approximate training time for the RL agent. These calculations take into account the hyperparameters, timeline size, and the number of stocks, and are subsequently presented to the user as a report. The user reviews the report and decides whether to go ahead with training the RL agent. If they agree, they pay the training cost and the tool commences the training process. The tool provides an ongoing evaluation of training through various metrics (**5.4 Model Training**) and, upon completion, delivers a comprehensive evaluation of the test set (**5.6 Testing / Evaluation**).

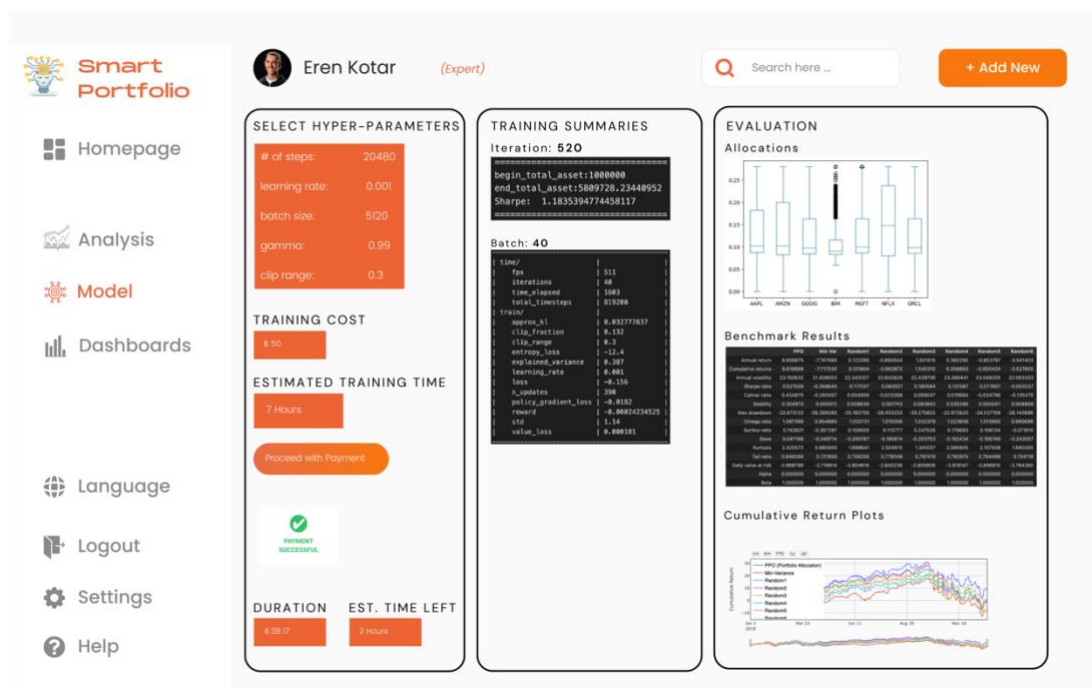


Figure 5.26 : Model page.

The trained model is stored in the user's database, and the user is informed about the pricing policies for using the RL agent. The user then decides whether to deploy the RL agent for real-time trading. If they proceed, they gain access to a portfolio dashboard where they can review profits, losses, costs associated with tool usage, and other relevant details.

The dashboard page (Figure 5.27) displays the customer's total revenue, along with parameters such as annualized return, sharpe ratio, volatility, and max drawdown. In addition, the customer will be able to see the stocks that the agent allocates on a daily basis, and the parts indicating the average and instant status of these allocations on the dashboard screen. A small section with actions has also been added to the dashboard.

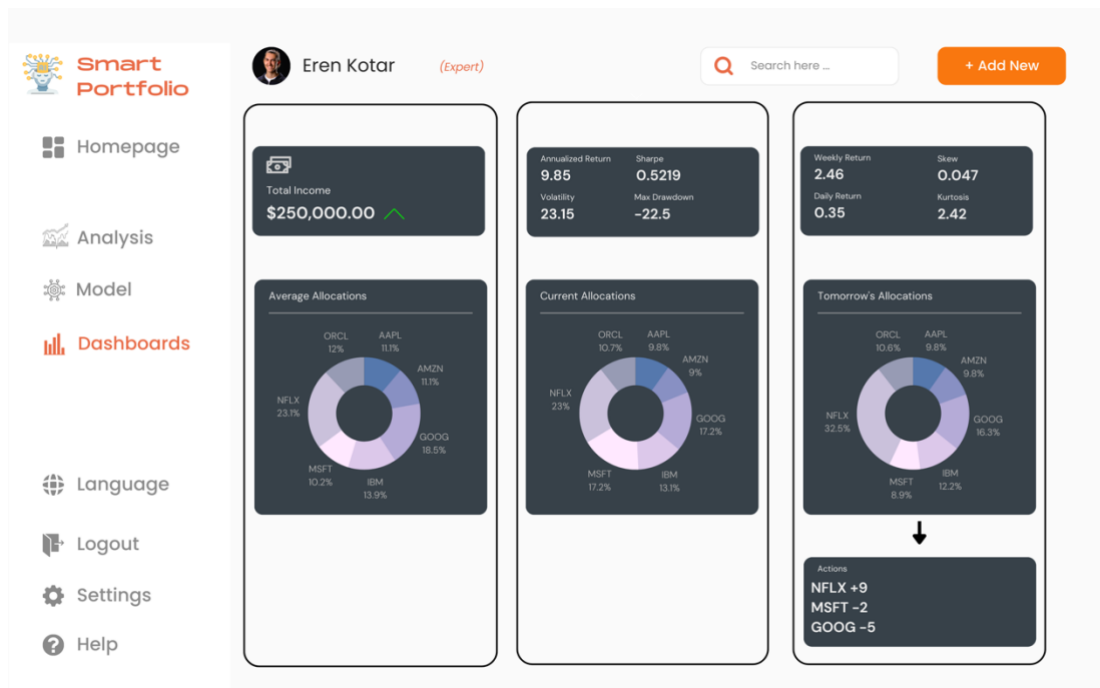


Figure 5.27 : Dashboard page.

6. CONCLUSIONS AND RECOMMENDATION

This graduation thesis focused on the development of a portfolio management tool for multi-stock trading using reinforcement learning techniques. The objective was to optimize the value of the customer's portfolio by accurately allocating the appropriate amounts to the portfolio assets, primarily consisting of stocks. Through experimentation, the algorithm demonstrated success in achieving this objective.

Additionally, this study provided a comprehensive analysis and evaluation of the RL algorithm itself which makes this tool unique and technical. By examining the outcomes and their implications, the study enhanced the understanding of the results and their significance for effective decision-making. The findings contribute to the field of multi-stock trading and reinforce the potential of reinforcement learning techniques in portfolio management.

Based on the results and insights gained from this thesis, several recommendations can be made for further development and application. Firstly, despite the demonstrated success of our current model, it's important to continue refining the algorithm to further improve its performance to changing market conditions.

Furthermore, incorporating additional factors and indicators, such as economic trends, companies' balance sheets and market analysis that have casual relationship with stock prices could provide further valuable insights for decision-making.

Moreover, expanding the scope of the tool to include other asset classes or investment strategies would offer more comprehensive portfolio management capabilities. Although our study has been largely algorithmic, improving the user interface for the portfolio management tool would increase its accessibility to a broader range of users. Efforts should be made to ensure that the tool is user-friendly, interpretable, and transparent in its decision-making processes.

In conclusion, this graduation thesis presents a valuable portfolio management tool for multi-stock trading, showcasing the potential of reinforcement learning techniques in

enhancing decision-making processes. Further research and refinement of the proposed framework hold promising prospects for advancing the field of portfolio management in the context of multi-stock trading.

7. REFERENCES

- Albadvi, A., Chaharsooghi, S. K., & Esfahanipour, A. (2007). Decision making in stock trading: An application of PROMETHEE. *European Journal of Operational Research*, 177(2), 673–683. <https://doi.org/10.1016/j.ejor.2005.11.022>
- Anderson. (1995). More effective time-series analysis and forecasting. *Journal of Computational and Applied Mathematics*, 64(1), 117–147. [https://doi.org/10.1016/0377-0427\(95\)00011-9](https://doi.org/10.1016/0377-0427(95)00011-9)
- Appel. (2008). Understanding MACD (Moving Average Convergence Divergence). *Traders Press, Inc.*
- Barfuss, W. (2021). Dynamical systems as a level of cognitive analysis of multi-agent learning. *Neural Computing and Applications*, 34(3), 1653–1671. <https://doi.org/10.1007/s00521-021-06117-0>
- Bishop. (1994). Neural networks and their applications. *Review of Scientific Instruments*, 65(6), 1803–1832. <https://doi.org/10.1063/1.1144830>
- Bishop. (2006). Pattern recognition and machine learning (Information science and statistics). *Springer-Verlag New York, Inc.*
- Brynjolfsson, & McAfee. (2017). The business of artificial intelligence. *Harvard Business Review*, 1–20.
- Busoniu, L., Babuska, R., Schutter, B., & Ernst, D. (2010). Reinforcement learning and dynamic programming using function approximators. CRC Press/Balkema.
- Chun. (2006). *Core Python Programming*. Pearson Education. <https://learning.oreilly.com/library/view/core-python-programming/0132269937/ch01.html#ch01lev1sec1>
- de Almeida, Reynoso-Meza, Teresinha, & Steiner. (2016). Multi-objective optimization approach to stock market technical indicators. *2016 IEEE Congress on Evolutionary Computation (CEC)*, 3670–3677. <https://doi.org/https://doi.org/10.1109/CEC.2016.7744254>

- Ghahramani. (2004). Unsupervised learning. *Advanced Lectures on Machine Learning*, 3176. https://doi.org/https://doi.org/10.1007/978-3-540-28650-9_5
- Goodfellow, Bengio, & Courville. (2016). Deep learning. *The MIT Press*.
- Güçkıran, K. (2020). *Pekiştirmeli öğrenme problemlerinde keşif ve genelleştirme yöntemleri* [Master's thesis].
- Guida, T. (2019). *Big data and machine learning in quantitative investment*. John Wiley & Sons, Inc.
https://dcu.primo.exlibrisgroup.com/discovery/fulldisplay?docid=alma991005342663007206&context=L&vid=353DCU_INST:VU1&lang=en&search_scope=MyInst_and_CI&adaptor=Local%20Search%20Engine
- Hao, X., Zhang, G., & Ma, S. (2016). Deep Learning. *International Journal of Semantic Computing*, 10(03), 417–439. <https://doi.org/10.1142/s1793351x16500045>
- Harmon, E., & Harmon, S. (1997). *Reinforcement learning: a tutorial*.
<https://apps.dtic.mil/sti/citations/ADA323194>
- Hochreiter, & Schmidhuber. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Jiang, T., Gradus, J. L., & Rosellini, A. J. (2020). Supervised Machine Learning: A Brief Primer. *Behavior Therapy*, 51(5), 675–687.
<https://doi.org/10.1016/j.beth.2020.05.002>
- Khand, Anand, Qureshi, & Katper. (2019). The Performance of Exponential Moving Average, Moving Average Convergence-Divergence, Relative Strength Index and Momentum Trading Rules in the Pakistan Stock Market. *Indian Journal of Science and Technology*, 12(26). <https://doi.org/10.17485/ijst/2019/v12i26/145117>
- Konda, & Tsitsiklis. (1999). Actor-Critic Algorithms. *Advances in Neural Information Processing Systems*, 12.
- Krogh, A. (2008). What are artificial neural networks? *Nature Biotechnology*, 26(2), 195–197. <https://doi.org/10.1038/nbt1386>

- Kumar, R., Ajjan, H., & Niu, Y. (2008). Information Technology Portfolio Management. *Information Resources Management Journal*, 21(3), 64–87.
<https://doi.org/10.4018/irmj.2008070104>
- LeCun, Bengio, & Hinton. (2015). Deep Learning. *Nature*, 521(7553), 436–444.
<https://doi.org/10.1038/nature14539>
- Lewis. (2016). *Deep Time Series Forecasting with Python: An Intuitive Introduction to Deep Learning for Applied Time* (p. 67).
- Li, & Bastos. (2020). Stock Market Forecasting Using Deep Learning and Technical Analysis: A Systematic Review. *IEEE Access*, 8, 185232–185242.
<https://doi.org/10.1109/access.2020.3030226>
- Liu, Xiong, Zhong, Yang, & Walid. (2018). *Practical Deep Reinforcement Learning Approach for Stock Trading*.
- Lo, A. W. (2002). The Statistics of Sharpe Ratios. *Financial Analysts Journal*, 58(4), 36–52.
<https://doi.org/10.2469/faj.v58.n4.2453>
- Lydon. (2010). *Buying and Selling by Investment Trends* (1st ed.). Pearson Education.
- Madani, Arnaout, Mofrad, & Arnaout. (2018). Fast and accurate view classification of echocardiograms using deep learning. *Npj Digital Medicine*, 1(1).
<https://doi.org/10.1038/s41746-017-0013-1>
- Manesh. (2020). Machine learning algorithms-a review. *International Journal of Science and Research (IJSR)*, 9, 381–386. <https://doi.org/10.21275/ART20203995>
- Martinsuo, M. (2013). Project portfolio management in practice and in context. *International Journal of Project Management*, 31(6), 794–803.
<https://doi.org/10.1016/j.ijproman.2012.10.013>
- Mitra, & Mitra. (2011). *The handbook of news analytics in finance*. Wiley.
<https://books.google.com/books?hl=tr&lr=&id=oL2XUi60pAAC&oi=fnd&pg=PR5&dq=Mitra+and+Mitra+2011+risk&ots=7Z31ncnKmX&sig=KtnkZ4M25EooTHLpx4do0Rhi97Q>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik,

- A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- Nuti, Mirghaemi, Treleaven, & Yingsaeree. (2011). Algorithmic trading. *Computer*, 44(11), 61–69.
- Osisanwo, Akinsola, Awodele, Hinmikaiye, Olakanmi, & Akinjobi. (2017). Supervised Machine Learning Algorithms: Classification and Comparison. *International Journal of Computer Trends and Technology*, 48(3), 128–138. <https://doi.org/10.14445/22312803/ijctt-v48p126>
- Pang, Nijkamp, & Wu. (2019). Deep Learning With TensorFlow: A Review. *Journal of Educational and Behavioral Statistics*, 45(2), 227–248. <https://doi.org/10.3102/1076998619872761>
- Patel, Shah, Thakkar, & Kotecha. (2015). Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268. <https://doi.org/10.1016/j.eswa.2014.07.040>
- Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, & Duchesnay. (2011). Scikit-learn: machine learning in python. *The Journal of Machine Learning Research*, 12, 2825–2830. <https://dl-acm-org.dcu.idm.oclc.org/doi/abs/10.5555/1953048.2078195>
- Russell, & Norvig. (1995). A modern, agent-oriented approach to introductory artificial intelligence. *ACM SIGART Bulletin*, 6(2), 24–26. <https://doi.org/10.1145/201977.201989>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- Shah. (2019). An Empirical Study on Options Trading Strategy Using “Commodity Channel Index” For NSE’s Nifty Options in India. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.3323746>

- Shavandi, A., & Khedmati, M. (2022). A multi-agent deep reinforcement learning framework for algorithmic trading in financial markets. *Expert Systems with Applications*, 118124. <https://doi.org/10.1016/j.eswa.2022.118124>
- Shrestha, Krishna, & von Krogh. (2021). Augmenting organizational decision-making with deep learning algorithms: Principles, promises, and challenges. *Journal of Business Research*, 123, 588–603. <https://doi.org/10.1016/j.jbusres.2020.09.068>
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of go with deep neural networks and Tree Search. *Nature*, 529(7587), 484–489. <https://doi.org/10.1038/nature16961>
- Silver, Hubert, Schrittwieser, Antonoglou, Lai, Guez, Lanctot, Sifre, Kumaran, Graepel, Lillicrap, Simonyan, & Hassabis. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144. <https://doi.org/10.1126/science.aar6404>
- Stewart, Piros, & Heisler. (2019). *Portfolio management : theory and practice*. John Wiley & Sons, Inc. <http://ebookcentral.proquest.com/lib/dcu/detail.action?docID=5741214>
- Sutton, & Barto. (1998). Reinforcement learning. *Journal of Cognitive Neuroscience*, 11(1), 126–134. https://d1wqtxts1xzle7.cloudfront.net/6985553/ivry_rev-with-cover-page-v2.pdf?Expires=1669670961&Signature=KrQEjctYPfhFSrlepL0rwj-PvLR31AQDDuT-r106tjQaM8~BOnbF4CtUrLhiIZ6eQRdhhZK-pNld~9SQZAljzXdViVjxrZr-ae1p-ikVNVHWUnneCcajTQAP9y4y5oVOBvkbNez~HBUhj5zoqJUfTCp2WbtKJ9p-2r17--Utdr5B243~t-xWFdx1aILUrRD78qDqdrtdidV0cZpvOSUTD~TpyGyry8y3u3fbGF68E84rGcEW-kNLIIBziAA9GtTcKdo1BFIXTTTCURM6b8KL1ikWEROGqlxWMUsBVhTQKYEr8UzyYjAVQPrvGyeIzD5KTDfRIv69rIbfMInPafScA__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA

- Sutton, R. S. (1992). Introduction: The challenge of reinforcement learning. *Machine Learning*, 8(3-4), 225–227. <https://doi.org/10.1007/bf00992695>
- Sutton, R. S., & Barto, A. (2020). Reinforcement learning: An introduction. The MIT Press.
- Théate, T., & Ernst, D. (2021). An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173, 114632. <https://doi.org/10.1016/j.eswa.2021.114632>
- Thomsett. (2019). *Understanding momentum in investment technical analysis : making better predictions based on price, trend strength, and speed of change* (1st ed.). Business Expert Press.
- Tsay. (2000). Time Series and Forecasting: Brief History and Future Research. *Journal of the American Statistical Association*, 95(450), 638–643. <https://doi.org/10.1080/01621459.2000.10474241>
- Uhlmann, Pontes, Laghmouchi, & Bergmann. (2017). Intelligent Pattern Recognition of a SLM Machine Process and Sensor Data. *Procedia CIRP*, 62, 464–469. <https://doi.org/10.1016/j.procir.2016.06.060>
- Wu, Y., Mansimov, E., Grosse, R. B., Liao, S., & Ba, J. (2017). Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation. In *Advances in Neural Information Processing Systems* (pp. 5279-5288).
- Zhang, & Sutton. (2017). A Deeper Look at Experience Replay. *ArXiv Preprint ArXiv:1712.01275*.
- Zhang, Q. (2001). Stock Trading: An Optimal Selling Rule. *SIAM Journal on Control and Optimization*, 40(1), 64–87. <https://doi.org/10.1137/s0363012999356325>

CURRICULUM VITAE



Name Surname : Eren Kotar
Place and Date of Birth : Bursa / 1999
E-Mail : kotar19@itu.edu.tr

EDUCATION :

- **B.Sc.** : 2023, Istanbul Technical University, Faculty of Management, Industrial Engineering Department

PROFESSIONAL EXPERIENCE AND REWARDS:

- **Black Kite** **Istanbul, Turkey**
Junior Data Scientist (NLP Researcher) Sep 2022 – Feb 2023
- **Dogus Technology** **Istanbul, Turkey**
Data Science Intern May 2022 – Sep 2022

CURRICULUM VITAE



Name Surname : İnci Kınalı
Place and Date of Birth : Istanbul / 1999
E-Mail : kinali19@itu.edu.tr

EDUCATION :

- **B.Sc.** : 2023, Istanbul Technical University, Faculty of Management, Industrial Engineering Department

PROFESSIONAL EXPERIENCE AND REWARDS:

- **Pirelli** **Istanbul, Turkey**
Commercial and Financial Planning and Controlling Intern Apr 2023 – Present
- **Prometeon TYRE Group** **Istanbul, Turkey**
Financial Planning and Controlling Intern Jun 2021 – Apr 2023